Development of an Embedded HPEC Telemetry Unit and Dashboard Integration for the TUKZIE Rev 0 Platform



Presented by:
Samson Okuthe
Student number: OKTSAM001

Prepared for:
Simon Winberg
Co-supervisor: Sampath Jayalath
Department of Electrical Engineering
University of Cape Town

Submitted to the Department of Electrical Engineering at the University of Cape Town in partial fulfilment of the academic requirements for a Bachelor of Science degree in Electrical and Computer Engineering.

October 11, 2026

Declaration

1. I know that plagiarism is wrong. Plagiarism is to use another's work and pretend that it is one's own.
2. I have used the IEEE convention for citation and referencing. Each contribution to, and quotation in, this report from the work(s) of other people has been attributed, and has been cited and referenced.
3. This report is my own work.
4. I have not allowed, and will not allow, anyone to copy my work with the intention of passing it off as their own work or part thereof.

Signature: 

Samson Okuthe (OKTSAM001)

Date: October 11, 2026

Word count of the body of the report (Chapters 1 to 7, text only, excluding headings, tables, figures, captions and code listings; measured with TeXcount on 11 October 2026): 11 118.

Terms of Reference

This project (SW-7) is to develop a high-performance embedded computing (HPEC) telemetry unit for the TUKZIE Rev 0 electric cargo trike, integrated with the vehicle's existing dashboard, capable of acquiring synchronised inertial, battery, positioning and camera data, characterising the vehicle's dynamic response at the edge, preserving telemetry under intermittent cellular connectivity, and presenting a minimal set of safety-relevant information to the driver on that dashboard.

Acknowledgments

I thank my supervisor, A/Prof. Simon Winberg, and co-supervisor, Sampath Jayalath, for their guidance, and the TUKZIE Flagship Programme and its vacation-work teams, whose earlier work on the vehicle's dashboard, sensing and drivetrain this project builds on.

Abstract

Draft abstract, to be revised once field results are available.

This report describes the development of an embedded telemetry unit for the TUKZIE Rev 0, a 72 V electric cargo trike, and its integration with the vehicle's existing dashboard. An ESP32-S3 with an integrated LTE modem acquires data from two inertial measurement units at 200 Hz, the vehicle's battery-management system over Bluetooth Low Energy, and GNSS, computes time-domain ride-characterisation features at the edge, logs telemetry locally, and publishes it over MQTT. A Raspberry Pi 4, time-synchronised to the inertial stream by a pulse wire, runs a pretrained object detector that reports road users in coarse distance bands, with three time-of-flight sensors for close range, and a page written for the existing dashboard shows its live view and alerts in place of the windshield head-up display in the original brief. Bench validation to date shows fixed-rate 200 Hz acquisition with no dropped samples, per-sensor calibration reducing an 8% disagreement between nominally identical accelerometers to within measurement noise, GNSS positions a median of 2.7 m from a reference, automatic recovery of the cellular link within about 5 s of the signal returning, a complete read-back of the local log, and a median of 102 ms from camera readout to detection at 19.5 frames per second. On-vehicle field testing, which the ride-characterisation research questions and the glance-time question depend on, is the principal outstanding work.

Contents

1 Introduction **1**

1.1 The vehicle and the problem 1

1.2 Deliverables and scope change 2

1.3 Research questions 2

1.4 Engineering requirements and acceptance criteria 2

1.5 Scope and limitations 2

1.6 Report outline 4

2 Literature Review **5**

2.1 Scope and evaluation framework 5

2.2 Electric cargo trikes and the case for telemetry 5

2.3 Road roughness and ride response 5

2.4 Inertial signal processing and vibration separation 6

2.5 Edge computing and real-time telemetry architecture 6

2.6 Driver-display design 7

2.7 Cellular synchronisation and data integrity 7

2.8 Proximity sensing on low-speed vehicles 8

2.9 Synthesis and research gap 8

3 Methodology **9**

3.1 System overview and architecture selection 9

3.2 Hardware 11

3.3 Firmware architecture 11

3.4 Ride-characterisation signal processing 12

3.4.1 Per-sensor calibration 13

3.4.2 Speed-normalised index 13

3.4.3 Dual-IMU fusion 14

3.5 Battery-management interface 14

3.6 Positioning 14

3.7 Other vehicle inputs 15

3.8 Camera subsystem 15

3.8.1 Capture and synchronisation 15

3.8.2 Detection 16

3.8.3 Range sensing and fusion 16

3.8.4 Dashboard integration 18

3.9 Communications and local logging 19

3.10 Verification 19

3.10.1 Start-up and interface checks 19

3.10.2 Integration and power 20

3.10.3 Vehicle tests 20

4 Results **22**

4.1 Inertial acquisition (R1) 22

4.2 Calibration (R2) . . . 23
4.3 Ride features and fusion (R3) . . . 23
4.4 Position (R4) . . . 23
4.5 Battery telemetry (R5) . . . 25
4.6 Hazard sensing (R6 to R8) . . . 25
4.7 Telemetry, logging and recovery (R9, R10) . . . 26
4.8 Driver display (R11, R12) . . . 26
4.9 Integration and power . . . 27
4.10 Summary against the requirements . . . 27

5 Discussion **28**

5.1 Acquisition integrity and what it costs . . . 28
5.2 Ride characterisation (RQ1, RQ4) . . . 28
5.3 Hazard sensing . . . 29
5.4 Data integrity and connectivity (RQ3) . . . 29
5.5 Driver display (RQ2) . . . 29
5.6 Integration and power . . . 30
5.7 Validity and limitations . . . 30

6 Conclusions **31**

7 Recommendations **33**

A Graduate Attributes, AI Use and Project Repository **39**

A.1 Graduate attribute evidence . . . 39
A.2 Project repository . . . 41

A.3 Declaration of Artificial Intelligence Use 41

B Supporting Figures **42**

C Wiring and Connections **45**

C.1 Pin references 45

C.2 Raspberry Pi 4 to the time-of-flight sensor board 47

C.3 Telemetry unit (ESP32-S3) to the Raspberry Pi 4 47

C.4 Telemetry unit sensors and antennas 48

C.5 Camera, displays and power 49

C.6 Wiring checks 49

D Test Evidence **50**

E Configuration and Development Record **56**

E.1 Firmware configuration 56

E.2 Development faults 57

E.3 Supporting detail 60

Chapter 1

Introduction

1.1 The vehicle and the problem

The TUKZIE Flagship Programme is developing an energy-aware, secure and autonomy-ready platform for last-mile logistics and urban freight, a market that is growing as cities grow [1]. Its testbed is the TUKZIE Rev 0, a 72 V electric cargo trike manufactured by Jinpeng [2]. Inspection of the motor and controller nameplates recorded a three-phase, permanent-magnet, sensored hub motor rated at 72 V DC and 35 A with a controller current limit of 90 A, about 2.5 kW of rated input power [3]; the vehicle's own plate (Figure B.1, Appendix B) gives model L2e-U, a rated output of 3.9 kW and a rated speed of 42 km/h. Electric tricycles of this kind already carry urban goods in African cities, where one study found operating costs per kilometre 45.5 to 86% below those of motorcycles, petrol tuk-tuks and a light car on the routes it measured [4].

At the start of this project the vehicle had a touchscreen dashboard on a Raspberry Pi 5, written by earlier vacation-work teams, but no live data: every page was driven by simulated values (Section 3.8.4). Nothing measured how the trike responds to the road and its own powertrain, the battery state was not read, and there was no record of a journey. A vehicle intended for fleet use and later autonomy needs all of these, and it needs them on a platform that vibrates strongly, has limited power and computing, and loses cellular coverage. Road traffic injuries are also a leading cause of death, mostly in low- and middle-income countries [5], so hazard alerts for the driver are worth having, provided that the display does not itself become a distraction: NHTSA guidance limits single glances away from the road to 2 s and a task to 12 s in total [6].

The engineering problem is therefore to acquire synchronised data reliably on a high-vibration, power-constrained vehicle; to distinguish road-induced motion from vehicle-specific dynamics; to keep processing and display latency bounded; to preserve telemetry through intermittent connectivity; and to present the result to the driver without obstructing the view or making safety-relevant behaviour depend on a remote service [7].

1.2 Deliverables and scope change

The brief [7] asked for: (1) an embedded high-performance embedded computing (HPEC) telemetry unit with a deterministic multi-threaded architecture for sensor polling, signal processing, display rendering and networking; (2) a driver interface, originally a windshield head-up display (HUD); (3) synchronisation of telemetry with a central database over 4G/LTE; and (4) a technical report validating the data and the system. During the project the windshield HUD was replaced, with the supervisor's agreement on 1 October 2026 and a title change recorded by the course coordinator on 2 October, by integration with the vehicle's existing dashboard: that dashboard already carries the information a HUD would show, a second display would compete for the same attention [8], and the camera's hazard alerts serve the programme's move towards autonomy better than a second screen (Section 2.6). The camera hazard detector was added to the scope on the supervisor's direction, and time-of-flight proximity sensing after a later request for a live nearby-object alert.

The intended outcome was an integrated prototype demonstrated on the vehicle, Technology Readiness Level 6 [9], not a product qualified for unrestricted road use.

1.3 Research questions

RQ1 How accurately and repeatably can a low-cost edge-processing system characterise the trike's dynamic response to road- and powertrain-induced excitation in real time, using inertial sensors on a 72 V electric cargo trike?

RQ2 Which information hierarchy, layout and update strategy can present safety-critical telemetry and hazard alerts on the vehicle's existing dashboard while limiting visual clutter and driver distraction?

RQ3 What end-to-end latency, data-loss and recovery performance can be achieved across sensor acquisition, edge processing, local logging and 4G/LTE synchronisation?

RQ4 Which time- and frequency-domain features most effectively separate road-induced vibration from repeatable motor and powertrain vibration?

1.4 Engineering requirements and acceptance criteria

The research questions are answered through the engineering requirements in Table 1.1. Each requirement has an acceptance criterion fixed before the test, taken from the project's test procedures (`BenchTest/TEST_PROCEDURES.md`, test identifiers in the last column) or, for removability, from the vehicle integration and test plan (SW7-VIT-001). Chapter 4 reports each against its criterion (Table 4.4).

1.5 Scope and limitations

In scope and delivered on the bench: the ESP32-S3 telemetry unit (two IMUs, GNSS, battery-management system over Bluetooth Low Energy, LTE, local logging); a forward camera on a Raspberry

Table 1.1: Engineering requirements, acceptance criteria and the research question each serves.

ID	Requirement	Acceptance criterion	RQ	Test
R1	Fixed-rate dual-IMU acquisition above the 160 Hz ISO 2631-1 bound	Both IMUs at 195 Hz or more; zero dropped samples; with logging off no inter-sample gap over 10 ms, with buffered logging gaps over 10 ms only at a flush	1, 3	T1, T2, T9
R2	Per-sensor calibration	Resting magnitude 9.80 to 9.82 m/s^2 on both IMUs	1	T4
R3	Correct on-board features and fusion	Synthetic-sine self-test within 5%; all fusion self-tests pass; every fused window from both sensors	1, 4	T1
R4	Position	3D fix; parsed position within 10 m of a phone reference; no damaged reply accepted	1, 3	T3
R5	Battery telemetry	Connection and checksum-valid frames; pack voltage matches a meter	3	T7
R6	Camera to IMU time alignment	About 2 edges/s; median interval 500 ms; none outside 500 ± 20 ms	1	C6
R7	Hazard detection	Runs a full minute at 5 frames/s or more; person logged as person; alert onset (three frames plus latency) under about 0.5 s	2, 3	C1, C2, C8
R8	Proximity range	Three sensors answer and respond to the matching hand; within about 5% of a tape below 1 m; fused distance median error 5 cm or less at 0.5 and 1.0 m	2	C7, C9
R9	Store-and-forward telemetry	Acknowledged publishes; loss reported and reconnection without intervention after coverage returns; every stored record read back intact and parsing	3	T5, T6, T10
R10	Local dashboard link	One line per second, no missing sequence numbers, valid JSON; record under 3 s old at the dashboard; IMUs unaffected	2, 3	T11, T12
R11	Driver display shows only valid data	Only live values, "--" when missing, never simulated values; camera alerts listed and cleared; offline state shown	2	B2, B3
R12	Glance demand	Single glances within the 2 s NHTSA guidance [6]	2	on vehicle
R13	Removable installation	No drilling or permanent change; vehicle back to standard in about 20 min	all	VIT-001

Pi 4 with a pretrained object detector, time-synchronised to the inertial stream; three time-of-flight proximity sensors fused with the camera detections; and the integration of all of these with the existing PySide6 dashboard on the Raspberry Pi 5. In scope but not delivered by the time of writing: the Sensirion SEN55 air-quality sensor (received, cable outstanding, Section 3.7); the 8×8-zone VL53L5CX time-of-flight sensor (three VL53L0X sensors stand in); the motor Hall tap; and synchronisation with a central database (data reach only a public test broker). The windshield HUD is not built.

The limitations are: one vehicle and one set of hardware; a pretrained, general-purpose detector that reports object class, a coarse distance band, confidence, timestamp and alert status only, stores no frames, identifies no individual, does not track across frames and controls no vehicle function; and results that are, unless stated otherwise, from the bench.

1.6 Report outline

Chapter 2 reviews the literature on ride sensing, edge telemetry, driver displays and proximity sensing. Chapter 3 describes the final design and how each part was verified. Chapter 4 gives the results against the requirements, Chapter 5 interprets them, Chapter 6 answers the research questions and Chapter 7 sets out recommendations. Wiring, configuration, test evidence and the development record are in the appendices.

Chapter 2

Literature Review

2.1 Scope and evaluation framework

The system combines four fields that are usually studied apart: ride and road sensing, embedded signal processing, vehicular telemetry, and driver-display design. The literature is judged here by what the TUKZIE Rev 0 needs, not only by reported accuracy: computational cost, sampling requirements, sensitivity to speed and mounting, real-time latency, communication loss and driver workload. A technique that works on a passenger car, a smartphone or a simulator may not transfer to a light three-wheeled cargo vehicle.

2.2 Electric cargo trikes and the case for telemetry

Light electric freight vehicles can reduce the cost and emissions of urban delivery, but by how much depends on route, payload, prices and the comparison vehicle. In Dar es Salaam an electric cargo tricycle had operating costs 45.5% below a motorcycle and 86% below a light car on the routes studied [4]; in Portland, Saenz et al. found greenhouse-gas reductions of 51 to 72% for an electric-tricycle logistics service [10]. Both figures were obtained by measuring energy, distance and operations on the vehicles in service, and an operator can confirm such benefits for its own fleet only by measuring the same quantities. On-vehicle telemetry is the instrument for this, which is the link between the logistics literature and this project. A three-wheeled chassis also makes vehicle response an engineering concern: payload position, tyre pressure, speed and surface affect both stability and measured vibration, so operating state should be recorded with the inertial data, and passenger-car roughness thresholds should not be transferred directly.

2.3 Road roughness and ride response

The International Roughness Index (IRI) is computed from a measured road profile through a standard quarter-car model [11]. An accelerometer on a vehicle measures something else: the combined response of road, tyres, suspension, chassis, payload, speed and mounting. González et al. classified road profiles

from simulated vehicle accelerations [12], but the inference depends on the assumed vehicle model, which cannot be carried over to a trike. Smartphone and low-cost studies extend the response-based approach, and identify placement, orientation, speed, sampling rate and inconsistent ground truth as persistent sources of variation [13]; Alatoom and Smadi's microcontroller and single-board-computer framework is closer to this project because it allows controlled mounting and deterministic sampling [14]. None of these establishes a calibrated ride measure for a three-wheeled cargo vehicle. The defensible first claim for this project is therefore a repeatable, vehicle-specific response index and event log, validated by repeatability over traversals and sensitivity to speed and payload, not an IRI.

2.4 Inertial signal processing and vibration separation

An inertial signal contains gravity, orientation changes, road excitation, body motion, drivetrain vibration, noise and transients. Features from a nominally vertical axis are comparable only after orientation compensation, and become unreliable when samples are irregular or missing. Time-domain features (root-mean-square, standard deviation, peak-to-peak, crest factor, jerk, threshold counts) are cheap and interpretable; spectral energy and power spectral density separate sustained narrow-band vibration from broadband impacts. Bui et al. show rough-road detection running on an edge microcontroller [15]. Yiliguoqi et al. classified roughness with an SVM on vertical acceleration and roll features at only 10 Hz, correctly in 80 to 100% of trials across four vehicles [16]; that range is tied to their phones, routes, labels and thresholds and cannot be expected on the TUKZIE without retraining. Learned classifiers should therefore follow a transparent statistical baseline, not replace calibration.

Separating road excitation from powertrain vibration needs an empirical baseline: stationary runs at several motor speeds identify repeatable powertrain components, and coast-down and repeated route runs show which components remain without traction and which change with the surface. Windowed statistics should be normalised or stratified by speed, because the same road feature excites the vehicle at a different frequency at a different speed [17]. ISO 2631-1 defines the evaluation of whole-body vibration up to 80 Hz [18]; its weighted measures describe comfort, which should be reported separately from vehicle response and road geometry.

2.5 Edge computing and real-time telemetry architecture

Edge computing moves processing close to the sensors to reduce response time, bandwidth and dependence on a remote service [19], which suits a vehicle whose alerts must work without coverage and whose raw 200 Hz data would cost more to transmit than compact features. Embedded HPEC also allows several streams to be timestamped and fused before data leave the vehicle [20]. "High performance" here means predictable completion of time-critical work within a small power budget, not throughput. Threads alone do not give real-time behaviour: shared locks, blocking input and output, allocation and unbounded queues still cause missed samples or stale displays. A producer and consumer design with timestamped samples in bounded queues, explicit overload behaviour (an old display frame may be dropped, an inertial sample must not be lost silently) and reporting of jitter, queue occupancy, display age and end-to-end latency as distributions is therefore the appropriate pattern.

Pérez-González et al. give a close comparison: an electric-racing telemetry system with ESP32 acquisition, Raspberry Pi 4B analytics and LoRaWAN, reporting 99% packet delivery and sub-second latency on

their track [21]. The division between deterministic acquisition and higher-level processing supports the architecture used here; the radio result is specific to their gateway and route. LTE gives wider reach at the cost of subscription, coverage dependence and less predictable delay, which argues for local logging and asynchronous transmission whichever radio is used.

2.6 Driver-display design

Any display, head-down or head-up, draws the driver's eyes or attention from the road. NHTSA limits single glances to 2 s and a task to 12 s, and treats a vehicle stopped at a light as being driven [6]. A head-up position does not remove the problem: HUD use changes attention demand and driving performance [22], and head-down assessment methods do not fully capture HUD behaviour [23]. A windshield HUD also brings optical problems: a double image from the two glass surfaces that wedge glass corrects only for one viewing geometry, an eye box that must be measured per installation, and legibility that depends on ambient light [24, 25, 26]; misaligned or excessive graphics compete with hazards [27]. Each is harder on a removable, low-cost unit on a vibrating trike in daylight, and where a dashboard already carries speed and battery state a HUD adds a competing channel [8]. These are the reasons the project used the existing dashboard.

For a dashboard, the literature gives five design issues against which a page can be checked. Layout and clutter: symbol count was the most influential interface factor in a semi-automated driving study, which recommended fewer than six symbols at a time [28], and added channels raise mental demand [8], so a stable hierarchy (persistent speed and battery, brief prioritised alerts, navigation only when needed) is preferred. Legibility and contrast: ISO 15008 sets character height by visual angle, 20 arc minutes recommended and 16 acceptable, as reported in [29, 30]; at about 70 cm from a 10.1-inch, 1280 by 800 panel, 20 arc minutes is a capital height of about 24 pixels, and a humanist typeface cut total glance time by 10.6% among male drivers [29]. Interaction: touch keys improved performance up to about 17.5 mm [31], Euro NCAP's 2026 protocol sets a minimum touch-target size [32], and entering a destination took about 40 s on production systems [33], so such tasks belong to the parked state. Alert priority: safety alerts should be few, ordered and distinguishable from status. Data validity and update age: the reviewed guidance concerns legibility and workload and does not address stale data, but a display that keeps showing a value after its source has stopped presents an estimate as a measurement; this project therefore treats missing and out-of-date data as a display requirement in its own right (R11). Evaluation should combine technical measures (render time, update rate, contrast in representative light, age of the data shown) with glance behaviour and workload in controlled trials.

2.7 Cellular synchronisation and data integrity

Coverage and delay are outside the controller's control, so the real-time path must be separate from remote synchronisation, and each record should carry a monotonic timestamp, a session identifier, quality flags and a sequence number so that a server can detect gaps and duplicates. MQTT suits compact publish and subscribe telemetry and HTTP suits configuration and bulk upload, but neither ensures reliability by itself: a store-and-forward queue, bounded retries, acknowledgement and a reconnection test are needed, and the measures are local completeness, backlog, synchronisation delay, duplicates and recovery after an induced outage [7].

2.8 Proximity sensing on low-speed vehicles

Nearby-object alerting is treated as a requirement even for slow vehicles: De Simone et al. classified ultrasonic ranges for obstacle avoidance on retrofitted agricultural machines [34], and Noomwongs et al. combined RTK-GNSS with a 2D LiDAR on a low-speed micro electric vehicle, because vision alone needed computing such vehicles lack [35]. A multi-zone time-of-flight sensor such as the VL53L5CX resolves direction from one device; an independent characterisation found it near-linear against a white target but with a standard deviation near 50 mm beyond 400 mm and only 18.8 to 34.4% valid readings against a black target, and a warm-up drift of about 15 minutes [36]. Range on dark targets and in sunlight therefore has to be measured on the vehicle, not taken from the datasheet.

2.9 Synthesis and research gap

The reviewed work shows that low-cost inertial sensors can detect road anomalies, that edge devices can process sensor windows locally, and that in-vehicle displays can be read within glance limits when sparse and legible. It also shows three limitations: response-based roughness depends strongly on vehicle, speed, mounting and calibration; edge architectures are usually reported by accuracy or mean latency, without jitter, queue behaviour or outage recovery; and display guidance comes almost entirely from passenger cars.

The gap claim is bounded by the search made for it. An automated search of OpenAlex, arXiv and Crossref on a fixed query set (ride characterisation, three-wheeler dynamics, cellular vehicle telemetry, MQTT, IMU calibration and fusion, camera hazard detection on edge hardware, driver displays and BMS protocols), run three times from 19 September 2026 and holding 733 records (`Research/research.db`), and a targeted search of the three-wheeler market and open-source projects, found no system that integrates these concerns for a light electric cargo trike: fixed-rate dual-IMU acquisition with measured jitter and loss, on-board ride features, camera detection fused with time-of-flight ranging, LTE telemetry with local logging, and presentation on the vehicle's own dashboard with missing data shown as absent. The same search narrowed two earlier claims (`Research/gap_ledger.json`): open-architecture inertial, GNSS and cellular telemetry units already exist on two-wheeled racing vehicles, and local logging beside a cellular uplink appears in prior work, although its recovery is rarely measured. The contribution claimed is therefore the integration and its measurement on this vehicle class, not any single component.

Chapter 3

Methodology

This chapter describes the system as built and how each part was verified. Development followed a staged approach: each interface was tested on its own before higher-level functions were added, so that a fault could be traced to one change. The faults that shaped the design are given here as short findings; the full development record, with every fault, its cause and its fix, is in Appendix E, and the firmware configuration (versions, pins, bus speeds, task priorities, queue depths and partitions) is in Table E.1 of that appendix.

3.1 System overview and architecture selection

Figure 3.1 shows the system. An ESP32-S3 telemetry unit samples two inertial measurement units (IMUs), reads GNSS and the battery-management system (BMS), computes ride features, logs locally and publishes over LTE. A Raspberry Pi 4 at the front of the vehicle runs the camera detector, reads three time-of-flight sensors, fuses the two, and bridges the telemetry unit's data to the network. The vehicle's existing dashboard on a Raspberry Pi 5 reads both over Wi-Fi.

Table 3.1 compares the architectures that were feasible with the hardware available. The deciding requirements were R1 (a fixed 200 Hz sampling period with no loss) and R7 (camera detection at a useful frame rate). A microcontroller under FreeRTOS can hold a fixed sampling period but cannot run a camera network at a useful rate; a Raspberry Pi can run the network, but a general-purpose Linux scheduler gives no guarantee of a 5 ms period, so it was not relied on for acquisition. Splitting the work keeps each part where it is predictable. The camera went on a Raspberry Pi 4 rather than the Raspberry Pi 5 because the Pi 5 runs the inherited dashboard, and developing on it would have put a working deployment at risk. Power was not measured for any option; the one power finding is that the telemetry unit's modem must not be powered from a Raspberry Pi's USB port (Section 3.10.2).

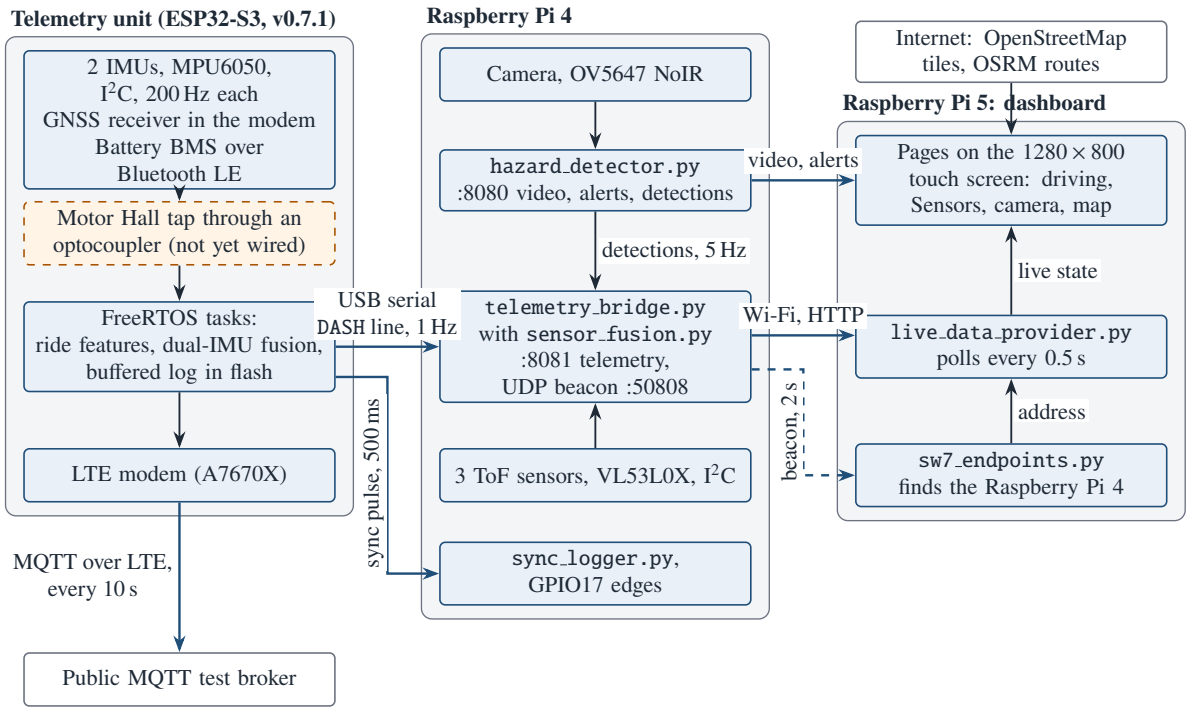


Figure 3.1: The system as built on 6 October 2026. Programs are named by their files in the repository. Dashed outline: built but not yet connected on the vehicle.

Table 3.1: Feasible architectures compared. Figures are measured in this project unless marked; power was not measured.

Option	**Workload**	**Power**	**Memory**	**Interfaces**	**Latency, timing**
ESP32-S3 alone	Dual 200 Hz sampling, features, BLE, modem: yes. Camera network at a useful rate: no	Lowest of the options (not measured)	16 MB flash, 8 MB PSRAM (not used)	Two I²C, UARTs, BLE, native USB; LTE and GNSS on the board	5 ms period held, 200.0 to 200.5 Hz
Raspberry Pi alone	Detector at 19.5 frames/s (Pi 4); fixed-period sampling not guaranteed	Higher; under-voltage on a weak supply	Not a constraint in testing	I²C, CSI camera, Wi-Fi, USB; no LTE modem	Detector 102 ms median; sampling jitter not bounded by design
ESP32-S3 and Pi 4 (chosen)	Acquisition on the ESP32, vision and fusion on the Pi	Two supplies; modem not from the Pi's USB	As above	Pulse wire for timing, USB serial for data	5 ms acquisition; 102 ms detection; 1 Hz dashboard line
Pi 5 with Hailo-8 accelerator (not tested)	Detector at camera rate expected; no Pi 4 link needed	Not measured	16 GB	Accelerator not yet enabled on the PCIe bus	Not measured

3.2 Hardware

The telemetry unit is a Makerfabs ESP32-S3 4G LTE CAT1 A7670X board: an ESP32-S3-WROOM-1-N16R8 module (16 MB flash, 8 MB PSRAM) with an A7670X LTE Cat-1 modem that also contains the GNSS receiver [37] (Figure B.2, Appendix B). The integrated board lets processor, modem and GNSS be tested without a separate modem circuit. Two MPU6050 IMUs are on separate I^2C buses at 400 kHz, which avoids their shared address and bus contention. The modem is on a hardware UART at 115200 bit/s, with its power key and reset on GPIO 4 and 5. The board's microSD slot was not used: its chip-select line passes through a resistor (R23) that the schematic marks as not fitted, so local logging uses the internal flash (Section 3.9).

The Raspberry Pi 4 carries an OV5647 camera module without an infrared-cut filter on a 15 cm ribbon, three VL53L0X time-of-flight sensors on its I^2C bus, and the sync-pulse input on GPIO17. The camera is housed in a three-part casing designed and 3D printed in this project (Figure B.5, Appendix B), not yet fixed to the vehicle. Every connection is listed in Appendix C, and Figure 3.2 shows the bench setup.

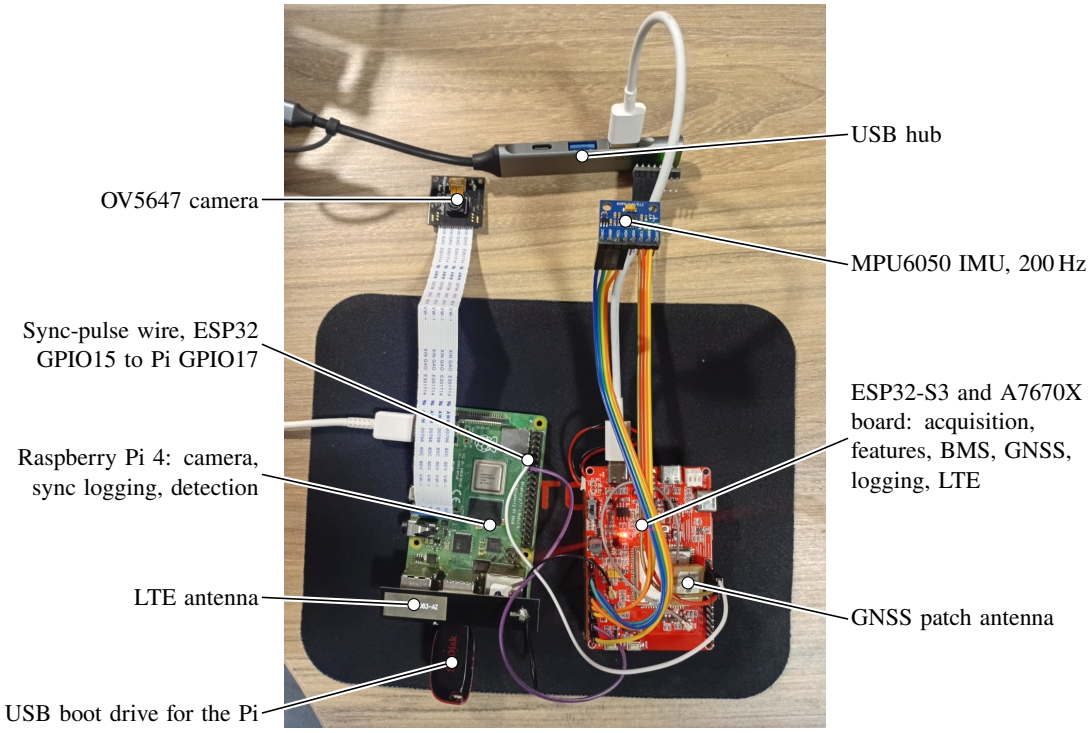


Figure 3.2: Bench setup of the telemetry system, each component labelled by its role.

3.3 Firmware architecture

The firmware is written with PlatformIO and the Arduino framework on FreeRTOS (Table E.1). Figure 3.3 shows its tasks and queues. The two IMU tasks run at the highest priority on core 1, each holding a 5 ms period with `vTaskDelayUntil`, which schedules each wake-up from the previous target time so that variable execution time does not accumulate into drift. The modem, BMS and dashboard-line tasks run on core 0, away from acquisition.

The 200 Hz rate follows from ISO 2631-1, which weights vibration up to 80 Hz [18]: sampling must

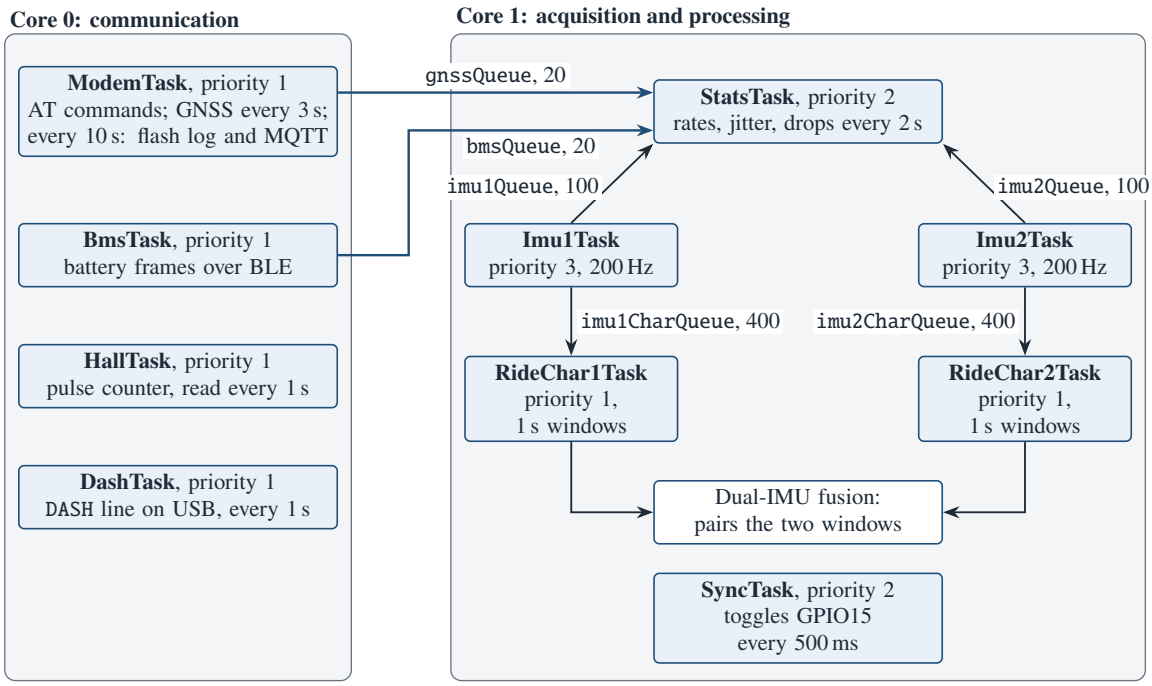


Figure 3.3: FreeRTOS tasks and queues (drawn for firmware v0.7.1; unchanged in v0.7.3). Priorities as set in the firmware (higher runs first); each queue is labelled with its name and capacity in items. Queues between the two cores are drawn in blue.

exceed 160 Hz, and 200 Hz adds margin without loading the bus heavily. Each sample carries a sensor identifier, a per-sensor sequence number and a timestamp from `esp_timer_get_time()`, a monotonic microsecond clock unaffected by network time. No acquisition task prints, because serial output blocks for a variable time and would disturb the period being measured. When a queue is full the producer drops the new sample and counts it rather than blocking, so a slow consumer can never hold up sampling. Each IMU feeds two queues, one to the statistics task and one to its ride-characterisation task, so that adding processing cannot change the acquisition accounting.

A lower-priority statistics task drains the queues and reports, every 2 s and per sensor, the achieved rate, the minimum, mean and maximum interval between samples and the drop count, with the GNSS poll and fix counts. These counters produce the acquisition results of Chapter 4 without a separate measurement pass, on the bench or on the vehicle.

Each IMU is set to $\pm 8\,g$, $\pm 500°/s$ and its widest digital low-pass filter, 260 Hz, so that the 80 Hz band of interest is not attenuated. Two limits follow: samples at the $\pm 8\,g$ limit are not flagged, so a clipped window under-reports its peaks; and 260 Hz is above the 100 Hz Nyquist frequency, so vibration between 100 and 260 Hz can alias into the band. The next filter setting, 94 Hz, limits aliasing at a small cost in the 80 Hz band.

3.4 Ride-characterisation signal processing

A task per IMU computes time-domain features from each one-second window. Features use the acceleration magnitude rather than one axis because the mounting has not been related to a vehicle frame; the magnitude does not depend on orientation, at the cost of not saying which axis a disturbance

came from. Each window holds $N = 200$ samples ($f_s = 200\,\text{Hz}$, $\Delta t = 1/f_s$). The calibrated magnitude of sample k is

$$a_k = s\sqrt{a_{x,k}^2 + a_{y,k}^2 + a_{z,k}^2}, \tag{3.1}$$

where s is the sensor's calibration factor. From the window,

$$\mu = \frac{1}{N}\sum_{k=1}^{N} a_k, \qquad a_{\text{RMS}} = \sqrt{\frac{1}{N}\sum_{k=1}^{N} a_k^2}, \qquad \sigma = \sqrt{\frac{1}{N}\sum_{k=1}^{N} a_k^2 - \mu^2}, \tag{3.2}$$

$$a_{\text{pp}} = \max_k a_k - \min_k a_k, \qquad \text{CF} = \frac{\max_k a_k}{a_{\text{RMS}}}, \qquad \bar{J} = \frac{1}{N-1}\sum_{k=2}^{N} \frac{|a_k - a_{k-1}|}{\Delta t}, \tag{3.3}$$

and the threshold count is the number of samples with $a_k > \mu + 2\sigma$. Because a_k includes gravity, a_{RMS} is close to $9.8\,\text{m/s}^2$ at rest and the crest factor close to 1; σ removes the constant part and is the vibration measure used. Lost samples are counted, not interpolated. The ISO 2631-1 frequency weighting is not implemented, so the features describe vibration exposure, not comfort as the standard defines it. At every boot the feature code is checked against a synthetic 5 Hz sine of 200 samples, whose RMS, peak-to-peak and crest factor have closed-form values ($A/\sqrt{2}$, $2A$, $\sqrt{2}$), with a tolerance of 5%.

3.4.1 Per-sensor calibration

Two nominally identical sensors at rest on the same platform read 10.35 to 10.39 and 9.55 to $9.56\,\text{m/s}^2$, about 5.7% above and 2.6% below standard gravity, a disagreement of about 8%. Since only the magnitude is used, one scale factor per sensor corrects it: RMS, σ, peak-to-peak and jerk scale linearly with it, and crest factor and the threshold count are unaffected. The first time a sensor is found with no stored factor, the firmware averages 200 stationary readings and stores

$$s = \frac{g_0}{\frac{1}{200}\sum_{k=1}^{200}\sqrt{a_{x,k}^2 + a_{y,k}^2 + a_{z,k}^2}}, \qquad g_0 = 9.80665\,\text{m/s}^2, \tag{3.4}$$

in non-volatile storage, rejecting values outside 0.5 to 2.0 as evidence that the sensor was moving. The factor is reused at every boot rather than recomputed, since the vehicle may be powered while moving, and a console command erases it so the sensors can be recalibrated in their mounted position. A single factor corrects the reading only in the orientation in which it was measured, because per-axis offsets remain; this is why the factors changed between boards (Chapter 4).

3.4.2 Speed-normalised index

The same road driven faster gives a larger response. Following Li et al., who classify roughness from acceleration mean square divided by speed [17], each window also computes

$$R = \frac{\sigma^2}{v}, \qquad v \geq v_{\min}, \tag{3.5}$$

where v is the latest GNSS speed and $v_{\min} = 1$ in the receiver's units; without a fix or below $v_{\min}$, R is not computed. Firmware before v0.7.2 used a_{RMS}^2, which is dominated by gravity; v0.7.2 uses the variance. Two limits apply: the receiver's speed units are not yet confirmed (Section 3.6), so R supports relative comparison within one dataset only; and the sensors are on the body, not the unsprung mass, so R is a

speed-normalised response measure, not a validated roughness class.

3.4.3 Dual-IMU fusion

The two sensors are the same part, configuration and calibration, so they are combined with equal weights: for RMS, σ, peak-to-peak, jerk and R, $x_f = (x_1 + x_2)/2$. Crest factor and the threshold count stay per sensor. Because the sensors are at different points on the vehicle, averaging would hide real differences, so each fused window also reports

$$D = \frac{|\sigma_1 - \sigma_2|}{\max(\sigma_f,\ \sigma_{\text{floor}})}, \qquad \sigma_{\text{floor}} = 0.05\,\text{m/s}^2, \tag{3.6}$$

which approaches 2 when one sensor sees much more than the other. A pair is fused only if the two windows ended within half a window of each other; if one sensor produces nothing for 3 s, the output falls back to the other and says so. A boot self-test checks the averaging, D, the rule that R needs both sensors, rejection of a misaligned pair and the fallback. The equal-noise assumption has not been tested on the vehicle.

3.5 Battery-management interface

The pack reports voltage, current, state of charge, capacity, temperatures and protection faults over Bluetooth Low Energy in the JBD protocol. An inherited implementation leaked a client object on every reconnection, reconnected on a fixed timer and framed messages by a terminator byte that can occur inside the data. The interface was rewritten with one reused client, reconnection only on a disconnect callback, framing from the declared length field, a checksum, and a plausibility check against a 72 V pack. It searches every discovered service for the required characteristics instead of assuming the service, and logs every advertiser seen so that a wrong address is visible. The field offsets were confirmed byte for byte against an independently written parser in a teammate's code, which does not check the checksum.

Current status (11 October 2026). The checksum formula, taken from community reverse engineering, included the register byte, so every genuine frame failed by exactly three; firmware v0.5.4 corrected it. On the vehicle on 30 September the unit then stored 14 consecutive checksum-valid records over about 140 s (74.78 V, 0.00 A, 60%) before the connection was lost. The link is intermittent: the pack's signal was −88 to −93 dBm, near the limit for holding a connection, so the unit may have to be mounted close to the pack. The reading has not been compared with a meter or the pack's app. The pack advertises as `DB24SA01L24S150ABU`, a 24-cell, 150 A designation, while inherited documentation describes the main traction pack as lead-acid without a BMS, so which pack this is remains to be confirmed with the vehicle team. The earlier connection faults are in Table E.2.

3.6 Positioning

The modem's GNSS is polled every 3 s with `AT+CGNSSINFO`. The parser was written defensively, checking the field count and logging the raw reply, because published reply formats differ between modem firmware versions [38]. That paid off: the assumed 13-field reply was in fact 9 empty fields indoors, and on a

rooftop on 29 and 30 September a fix gave 17 fields (and an 18-field variant while settling) in decimal degrees, so the first mapping had read satellite counts as the position. Since v0.5.3 the parser anchors on the latitude hemisphere letter and reads every field relative to it, which fits all observed layouts, and accepts a fix only if the layout and the coordinate, date and time formats are valid and the implied jump from the previous fix is under 100 m/s; this rejected replies corrupted when a second query competed with the firmware's polling. A console command prints every raw reply so that the parser can be checked without extra commands. The same session found that the modem's power key toggles the modem, so the boot-time pulse switched a running modem off after every reprogramming; since v0.5.2 the key is pulsed only if the modem does not answer.

Firmware v0.7.3 adds assisted GNSS, which downloads orbit data over LTE every two hours (`AT+CAGPS` returned success on the bench), and, while there is no fix, the modem's network position (`AT+CLBS`) once a minute. Indoors this answered in about 0.7 s with a reported accuracy of 550 m, about 30 m from the first outdoor fix at the same site; it is kept in separate fields, drawn on the map as an approximate circle, and never used for speed, heading or the ride index.

3.7 Other vehicle inputs

The Sensirion SEN55 reports particulate matter, humidity, temperature and VOC and NOx indices over I^2C; the gas indices are relative to recent background and take up to 1 and 6 h respectively to meet specification after switch-on [39]. It has been received but is not connected, for want of a six-way JST GH cable; the planned wiring is in Appendix C.

Motor speed would help separate powertrain vibration (RQ4). The Hall signal is referenced to the drivetrain, so it is to be tapped through a PC817 optocoupler module. Firmware v0.6.0 counts both edges in the hardware pulse counter with a 12.5 μs glitch filter, so counting costs the IMU tasks nothing, and computes revolutions per minute as the edge rate divided by twice the pole-pair count, which is not yet known. Until the tap is wired the counter reads the sync pulse as a loopback. The same firmware publishes the modem's signal quality every 10 s so that failed publishes can be related to coverage.

3.8 Camera subsystem

3.8.1 Capture and synchronisation

The camera and the inertial data are on different boards, so their clocks are related by a shared signal: a FreeRTOS task on the ESP32 toggles GPIO15 every 500 ms with `vTaskDelayUntil` and logs each edge's microsecond time, and the Raspberry Pi 4 logs the arrival of each edge on GPIO17. The edge logger (`CameraDetection/sync_logger.py`) takes each edge's time from the kernel through the GPIO character device, on the same monotonic clock as the camera's frame timestamps; the camera library's `SensorTimestamp` gives the time at which the first pixel of each frame was read out, on that clock [40]. Frames and edges therefore share one time base. Two findings shaped this part. The pin first chosen for the pulse is not brought out on the board, which was caught against the manufacturer's schematic before any wiring. And twice the jumper on the pulse wire was disturbed by handling, once giving no edges and once a 16 s burst of spurious edges: edges that do not fit the known 500 ms pattern can be discarded

in analysis, but a secured connector is needed on the vehicle. Before kernel timestamps were used, a lens-cover test bounded the offset between video and wall clock to under about 1 s, a bound set by the human reporting method.

3.8.2 Detection

The detector (`CameraDetection/hazard_detector.py`) runs a pretrained SSD-MobileNet-v1, trained on COCO and quantised to 8-bit integers for LiteRT, on 640×480 frames, which needs no training data and no accelerator. Only motor vehicles, motorcycles, bicycles, pedestrians and animals are reported, in three distance bands (immediate, warning, monitoring) estimated from the box width, an assumed width per class (for example 1.8 m for a car) and the pinhole relation. The focal length is not yet calibrated, so the bands are indicative.

An alert is raised when a class appears in a band for three consecutive frames, re-raised if it holds a nearer band for three frames, and cleared only after five frames without it; within a frame the nearest detection of a class is used. The clearing rule came from the second live run, in which an alert that cleared on a single missed frame produced 16 activations in 47 s for one person. The rules were checked offline by replaying scripted sequences through the unchanged loop, and later seen raising and clearing person alerts on live frames.

Data handling is minimal by design. Each alert records five fields: class, band, confidence, timestamp and alert status. Frames exist only in memory and are never written to disk; there is no face or number-plate recognition and no identification of anyone; inference is local; and the output is advisory, with no control of any vehicle function. The ride characterisation of Section 3.4 uses no learned model.

Every frame's latency is measured from its `SensorTimestamp`: the time from readout to capture by the detector, preprocessing, inference, alert logic and the total, reported as median, 95th percentile and maximum every 5 s and optionally written per frame. Inference uses all four cores. The optional live view draws the detections on the image and serves it as MJPEG from its own thread, which always takes the newest frame, so a slow viewer cannot slow detection. Two input faults were found on the first live run: a placeholder entry in the label file shifted every class by one, and the camera's buffers are in blue, green, red order while the model expects red, green, blue [40]. The module's lack of an infrared-cut filter gave a pink cast; its own tuning file with saturation 1.8 restored the colours, and a correction for near-black pixels is applied to the live view only (Appendix E).

Because the COCO classes include no road-surface hazards, a YOLO11n pothole model was fine-tuned in Colab, first on a public dataset and then on tiles cut from the road band of a South African dashcam dataset (Nienaber, Kroon and Booysen, Stellenbosch University); the data preparation is in Appendix E. *[Outstanding: run of the pothole model on the Raspberry Pi 4 beside the main detector, and its held-out test score.]*

3.8.3 Range sensing and fusion

A VL53L5CX was chosen for proximity sensing because its 8×8 zones in a 63° field separate left, centre and right from one device, where single-point sensors would have to be duplicated across directions. It involves no image and no learned model, so it works whatever the detector's state. It had not arrived

by 30 September, so the three VL53L0X sensors of a UCT micromouse sensor board stand in, facing left, ahead and right, with a range of about 2 m (1.2 m used). They share one address, so the reader holds all three in shutdown and wakes each in turn to give it its own address (Appendix C). They are on the Raspberry Pi 4, which sits beside the camera and has a free bus, leaving the ESP32's acquisition undisturbed.

Finding: one loose sensor stopped all three. On 6 October every distance was null, because the start-up stopped at the first sensor that did not answer. The reader now skips a sensor that does not answer and logs which one, the bridge looks for it again every 60 s, and since 10 October each I/O call has a 0.5 s timeout so that a stuck sensor cannot freeze the reader; all three sensors were reading afterwards (`Planning/REMAINING_WORK.md`).

The camera says what an object is and in which direction, but its distance is uncalibrated; each time-of-flight sensor measures distance well, but only close by, in one direction and without a class. A fusion step in the bridge (`CameraDetection/sensor_fusion.py`) combines them. The detector serves each detection with a bearing computed from the box centre and the 53.5° horizontal field, and the bridge polls it five times a second. A detection is matched to a sensor when its bearing lies in that sensor's cone widened by a 5° gate and the sensor's nearest sample is within 200 ms, valid and in range; the nearest reading wins and each sensor serves one detection. A matched detection becomes a fused hazard with the camera's class and the sensor's distance; an unmatched one keeps the camera estimate; a sensor with three valid in-range readings and no match reports an unclassified obstacle; and camera data older than 1 s are ignored, so with the detector stopped only obstacles are reported. Hazards are sorted nearest first. The sensor directions (−35, 0 and +35°, 12.5° half-angle) are defaults, not measurements, and with them the side cones overlap the camera's view only at its outer edges (Figure 3.4). A sensor reports the nearest surface in its cone, which need not be the object the camera saw, and there is no tracking. No log of distances and bearings is kept unless a validation session asks for one.

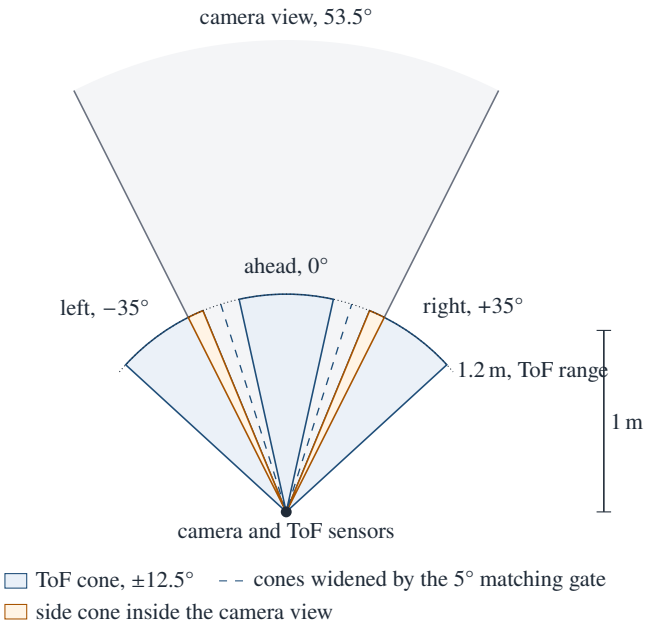


Figure 3.4: Camera and time-of-flight geometry used by the fusion, top view to scale, forward up. The values are the configuration defaults, not yet calibrated. A detection is matched to a sensor when its bearing lies within that sensor's cone widened by the 5° gate.

3.8.4 Dashboard integration

An inspection of the inherited PySide6 dashboard on 2 October found no live input: every page ran on simulated data, with one unused entry point for a live vehicle state and a fallback to simulation after 2.5 s without one. A driver's display should not depend on coverage or a remote broker, so a local path was built. Firmware v0.7.0 prints one compact JSON line a second on USB; a bridge on the Raspberry Pi 4 (`telemetry_bridge.py`) reads it with the time-of-flight distances and the fused hazards and serves them over HTTP; and a live-data provider on the Raspberry Pi 5 polls the bridge every 0.5 s. The dashboard therefore needs one network link for camera, alerts and telemetry. It finds the Raspberry Pi 4 on any network from a beacon that the bridge broadcasts every 2 s, with localhost, an override, a wired address, mDNS and the last good address as alternatives (Figure E.1, Appendix E). The design addresses the dashboard issues of Section 2.6 as follows.

Data validity and update age. A live-only mode (`TUKZIE_LIVE_ONLY=1`) stops the simulation entirely. When the bridge cannot be reached, or the telemetry unit's record is more than 3 s old, the provider stops sending, and within 2.5 s the dashboard shows an explicit no-data state: every sensor field empty, every validity flag false, "--" or "Unavailable" on the pages, and a message that live data was lost. A field with no live source (speed, until the GNSS units are confirmed; pedals; temperatures) stays empty rather than taking a default, and the state of charge stays empty until the BMS reports. This mode was added after two findings: when the Raspberry Pi 4 changed network, the dashboard fell back to simulated values with no sign, and a battery level of 82% was shown that was the simulator's last value, not a measurement.

Layout. The changes add few elements: a camera page with the live view and the active alerts; a Sensors page with the three distances as gauges, the fused hazards nearest first, and the battery, position, motor and ride readings; a ride-quality card on the diagnostics page; and the driving page's front-obstacle distance, taken from the forward sensor only so that a car parked alongside does not raise a front warning.

Contrast and colour. Alerts and gauges use the dashboard's own status colours (amber under 1.0 m, red under 0.5 m), so the team's visual language is kept. Contrast in daylight has not been measured.

Alert priority. An immediate hazard from the fusion, or loss of the Raspberry Pi 4 link for 5 s after it was working, raises a banner over every page; touching it opens the Sensors page, and a button hides it for 30 s. Lower bands appear only on the camera and Sensors pages.

Interaction. Finger-drag scrolling was added and developer options were moved behind one switch. Touch response was reported as slow, so the touch controller was read directly during 25 taps (Chapter 4).

The integration is applied as a patch set of 63 edits to a separate copy of the team's dashboard by a deploy script that refuses to touch the team's folder and writes nothing unless every edit's anchor text occurs exactly once, so the team's code is never changed. Since 6 October a system service starts the integrated copy at boot on the Raspberry Pi 5 in place of the team's service, which one command restores. The camera page uses only Qt's networking and widget modules, because the dashboard's web-engine map crashed the program on the Raspberry Pi 5; a street-tile map drawn directly with Qt replaces it. The map and navigation work is summarised in Appendix E.

3.9 Communications and local logging

Every 10 s the modem task publishes a compact JSON summary of the ride, BMS and GNSS state over MQTT using the modem's own `AT+CMQTT` commands. Publishing is kept inside the modem task because the modem UART has no lock, and a second task issuing commands would corrupt both streams. The broker is a public test broker (`test.mosquitto.org`), used only to prove the path. For the start, connect and publish commands the modem returns `OK` and then a separate result line whose code gives success or failure [38]; since v0.5.0 the firmware waits for that line rather than trusting the `OK`. A session is treated as lost after three failed publishes or when the modem reports a dropped client or lost network; the old session is then shut down in the order the manual requires, and retries start at 30 s and double to at most 5 minutes. Since v0.5.3 the firmware also reconnects as soon as the modem reports that packet data is available again. Console commands report the state, switch logging off and on, force a reconnection, dump or clear the log, and erase the calibration.

Each record is also written to a LittleFS file system in the 3.5 MB data partition of the internal flash (Table E.1), whether or not MQTT is connected, so that a session leaves a complete record without coverage. Two findings shaped this part.

Finding: flash writes pause acquisition. With a write every 10 s, both IMUs showed a 33 to 40 ms gap at each write, although they are on separate buses; switching logging off for 120 s, with publishing continuing, removed the gaps. Writing to flash suspends code executing from flash on both cores, which explains why both stalled together. Since v0.6.0 records are buffered in memory (8 KB) and written in one append about once a minute. This reduces how often the pause occurs, not its length, and up to a minute of records is lost if power is cut before a flush.

Finding: missing values made invalid JSON. Reading the whole log back through a paced, indexed dump (`BenchTest/log_dump.py`) showed that most records contained `nan` for fields with no value, such as the position before a fix, which JSON does not allow; the same text had been published. Watching the live stream had not shown it. Since v0.6.3 such fields are written as `null`.

The log is capped at 3 MB and then truncated rather than rotated, and it does not yet record which lines have been published, so the backlog is not replayed after an outage. *[Outstanding: central database ingest on the Raspberry Pi 5, sequence numbers and backlog replay (firmware v0.7.4), with the synchronisation measurements: completeness, delay and duplicates.]*

3.10 Verification

3.10.1 Start-up and interface checks

The routine checks that establish each interface before the higher-level tests are consolidated in Table 3.2. They are pass or fail, and all passed; the faults they exposed are in Table E.2. The acceptance tests of Table 1.1 are reported in Chapter 4.

Table 3.2: Start-up and interface checks (bench).

Check	Evidence required	Result
Processor boot	Versioned banner, continuous status, no reset loop; bounded wait for a USB host	Pass
Flash configuration	16 MB flash in DIO mode with the 16 MB partition table; LittleFS mounts	Pass after the DIO fix
Serial routing	Application output on the target board's native USB port	Pass after the routing fix
Modem power-up	Modem answers `AT` without the button; key pulsed only if silent	Pass (v0.5.2)
UART, SIM, signal	`OK` to `AT`; `+CPIN: READY`; valid `AT+CSQ`	Pass; CSQ 22 to 30
LTE registration	`AT+CEREG?` and `AT+CPSI?` registered, `LTE,Online`	Pass; intermittent indoors
GNSS interface	Correctly framed `+CGNSSINFO` after warm-up	Pass
Manual bridge	Commands and replies passed both ways	Pass
I^2C levels	Idle bus at about 3.3 V (inputs not 5 V tolerant)	Pass, 3.3 V
Both IMUs on target board	Found, configured, calibrated, 400 to 401 samples per 2 s	Pass, 0 dropped
Raspberry Pi 4	Same OS release as the Pi 5; camera identified (OV5647); network and ssh reachable	Pass
Pre-test check	`BenchTest/sw7_check.py` reports every link before a session	16 offline tests pass; READY on 4 Oct except the Pi 4 supply

3.10.2 Integration and power

All telemetry-unit subsystems were first run together on the target board, and then the whole chain from the telemetry unit through the Raspberry Pi 4 to the dashboard on the Raspberry Pi 5. Finding: the Raspberry Pi 4 ran under-voltage and throttled to 600 MHz through much of the bench work, and on 6 October stopped booting. A logger of its power and temperature flags every 2 s showed the flags clear with nothing attached, and under-voltage starting 14 s after the telemetry unit was plugged into its USB port, when the LTE modem came up. The modem's transmit current through the Pi's USB port is the cause, so the telemetry unit must have its own supply, for example a powered hub. The same log showed the Pi 4 reaching its soft temperature limit with the detector running and no cooling, and a fan was fitted (Appendix C).

3.10.3 Vehicle tests

The vehicle integration and test plan (SW7-VIT-001) divides the vehicle work into a stationary phase with the drive wheel raised and a phase with the vehicle on the ground. The stationary phase covers mounting, calibration in the mounted position, acquisition timing, synchronisation, the Hall tap and pole-pair count, a powertrain vibration baseline with the motor run at low, medium and high settings for about 60 s each, BMS voltage against a meter, GNSS and LTE static tests and an induced outage. The ground phase covers repeatability over repeated traversals, speed dependence, surface discrimination, sensor placement, coast-down separation of the powertrain, and payload. Sessions with the 72 V system energised require a second person, rated stands and chocks; all mounting is removable, with no drilling or permanent modification, and the vehicle can be returned to standard in about 20 minutes (R13). *[Outstanding: road test of 14 October 2026: protocol, route, conditions, mounting positions and the*

sessions completed.]

Chapter 4

Results

This chapter gives the measured results against the requirements of Table 1.1. Unless marked otherwise, every figure was measured on the bench, on the target board and the two Raspberry Pis, between 25 September and 11 October 2026; the raw evidence is indexed in Appendix D. Results that depend on the vehicle are marked as outstanding where they will go.

4.1 Inertial acquisition (R1)

Table 4.1 summarises the acquisition results. Both IMUs held 200 Hz with no dropped samples in every recorded session, including with all subsystems running together on the target board. The only timing fault was the pause at each flash write. With a write every 10 s (v0.5.0) both IMUs showed a 33 to 40 ms gap at every write, twelve in two minutes; with logging switched off for the next two minutes the worst gap was 5.9 ms. With buffered logging (v0.6.0), in a 330 s run with five flushes, all ten gaps over 10 ms (37 to 40 ms, both IMUs at each flush) fell within 3.5 s after a flush, and the worst gap elsewhere was again 5.9 ms. Each flush occupied the modem task for 71 to 202 ms, which does not affect acquisition. R1 is met on the bench.

Table 4.1: Inertial acquisition on the target board (bench).

Measure	**Result**
Sample rate, each IMU	200.0 to 200.5 Hz in every session
Dropped samples	0 in every session
Longest gap, logging off	5.9 ms; none over 10 ms
Gap at each flash write, v0.5.0	33 to 40 ms, every 10 s
Gap at each flash write, v0.6.0 (buffered)	37 to 40 ms, about once a minute, all within 3.5 s of a flush
All subsystems together (IMUs, BLE, modem)	400 to 401 samples per 2 s window on each IMU, 0 dropped
Dashboard line added (v0.7.0)	200 Hz on both IMUs, 0 dropped

[Outstanding: acquisition rate, drops and gap distribution on the moving vehicle (road test, 14 October 2026).]

4.2 Calibration (R2)

Before calibration, the two IMUs on the development board read about 5.7% above and 2.6% below standard gravity at rest, about 8% apart. The factors stored at their first calibration were 0.9773 and 0.9901, after which both read 9.80 to 9.81 m/s^2, agreeing within measurement noise. The factors imply mean magnitudes at the moment of calibration of 10.03 and 9.90 m/s^2, not the readings seen earlier that day, so the uncalibrated readings had shifted between sessions; the likely cause, not confirmed, is that the sensors were moved in between. On the target board the first boot measured 9.44 and 9.57 m/s^2 (factors 1.0390 and 1.0245), and with those factors a later session read median resting magnitudes of 9.93 and 10.10 m/s^2, 1.3% and 2.9% above gravity. R2 was met immediately after calibration but not in the later session, which confirms that a single factor holds only for the orientation in which it was measured.

[Outstanding: recalibration in the mounted position on the vehicle and the resulting resting magnitudes.]

4.3 Ride features and fusion (R3)

The feature self-test passed at every boot: RMS, peak-to-peak and crest factor of the synthetic sine were within 5% of their analytical values. All five fusion self-tests passed. In a 150 s recorded session (Figure 4.1) all 149 fused windows combined both sensors with a skew below 1 ms. While the board was still, the fused vibration level σ had a median of 0.041 m/s^2 and a maximum of 0.067 m/s^2; during the hand disturbance it rose to 9.35 m/s^2, and the largest peak-to-peak amplitude in a window was 77.6 m/s^2, within 1 m/s^2 of the $\pm 8\,g$ range, so that peak was probably clipped. The disagreement figure D ranged from 0.00 to 1.68. In an earlier disturbance test both sensors rose together (peak-to-peak from about 0.2 to 18.4 and 10.2 m/s^2, jerk from about 8 to 160 and 162 m/s^3) and decayed together. R3 is met. These results show that the features are computed and combined as designed; they do not show how the features respond to the road or the motor.

[Outstanding: ride features, speed-normalised index and disagreement on the vehicle: stationary powertrain baseline at each motor setting, repeated traversals of the same road sections, and speed dependence (RQ1, RQ4).]

4.4 Position (R4)

With the corrected parser, the receiver reached a three-dimensional fix 105 s after a cold start (51 s in an earlier session), with 23 to 25 satellites and a horizontal dilution of precision of about 0.9 to 1.5. Over 104 fixes the position was a median of 2.7 m from a phone's position, 95% within 11.0 m and at most 12.3 m away. The phone has an error of a few metres itself, so this is a consistency check, not an absolute accuracy; against the criterion of 10 m, the median and most fixes pass but the 95th percentile does not. The speed field read 0 to 5.8 while stationary, so its units and low-speed noise are unconfirmed. The network position (v0.7.3) answered in about 0.7 s with a reported accuracy of 550 m.

[Outstanding: time to first fix with assisted GNSS, and the speed field against a known speed on the vehicle.]

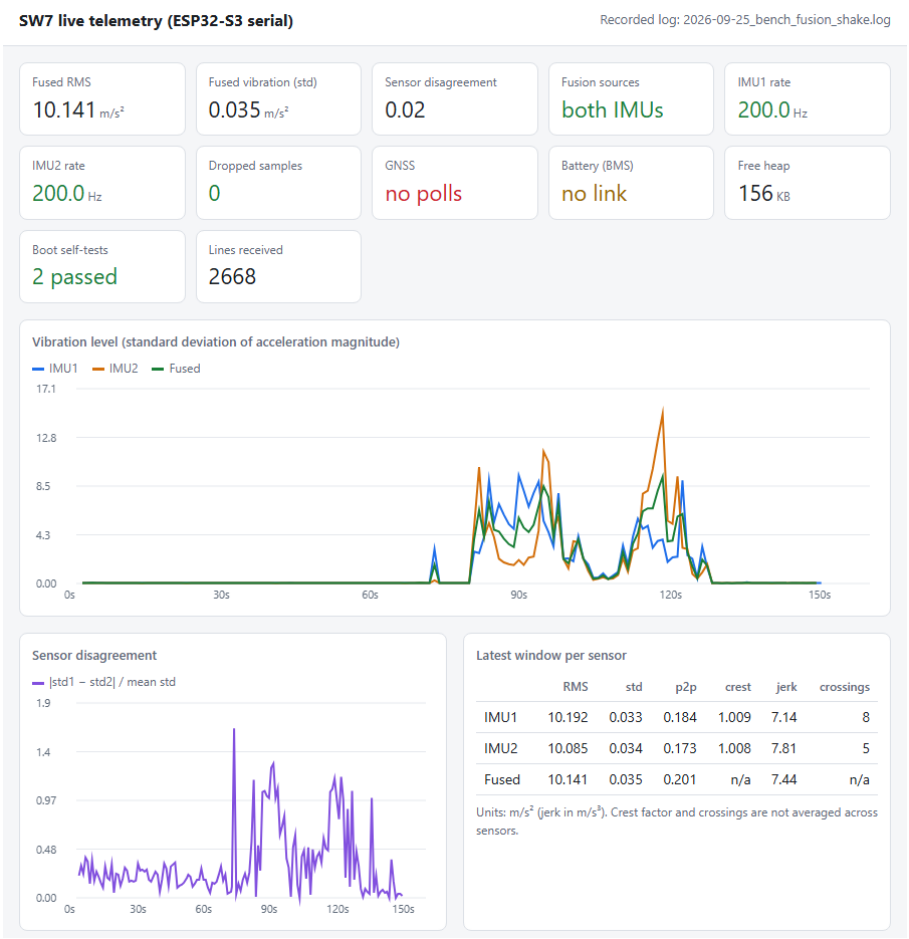


Figure 4.1: A 150-second bench session recorded from power-up and replayed through the bench dashboard: still until about 75 s, disturbed by hand until about 128 s, then still again.

4.5 Battery telemetry (R5)

After the checksum correction the unit stored 14 consecutive valid records on the vehicle over about 140 s: 74.78 V, 0.00 A and 60% state of charge (3.12 V per cell for 24 cells), at −88 to −93 dBm, after which the connection was lost. R5 is partly met: valid frames were received, but the connection is not sustained and the voltage has not been compared with a meter.

[Outstanding: sustained BMS session on the vehicle with the voltage checked against a meter.]

4.6 Hazard sensing (R6 to R8)

Table 4.2 summarises the hazard-sensing results and Table 4.3 gives the latency breakdown. With four inference threads the detector ran at 19.5 frames/s with a median of 102 ms from sensor readout to result. The largest share, about 52 ms, was the time from readout to the detector receiving the frame, which covers readout and the camera's image processing; inference took 36 ms. With one thread inference took 126 ms and the rate fell to 7.0 frames/s, as in the first live runs. The processor reached 65.7 °C without throttling. At 30 km/h, 102 ms is 0.85 m of travel; the three-frame onset adds about two frame intervals, so an alert is raised about 0.2 s, or 1.7 m, after an object first appears. A person was detected with confidence 0.50 to 0.73; only the person class has been confirmed on live frames. R6 and R7 are met on the bench.

Table 4.2: Hazard sensing on the Raspberry Pi 4 (bench).

Measure	**Result**
Camera to telemetry-unit alignment (C6)	Sync pulse median 500.06 ms, range 499.72 to 500.41 ms (121 edges, 60 s)
Detector rate (SSD-MobileNet, 4 threads)	19.5 frames/s; 17.3 with the live-view server on
Sensor readout to result	102 ms median, 120 ms 95th percentile, 136 ms maximum
Alert onset (3 frames)	about 0.2 s, about 1.7 m of travel at 30 km/h
Alert clearing	before the fix, 16 activations in 47 s for one person; after it, cleared only after 5 empty frames
Time-of-flight sensors (3 × VL53L0X)	about 31 readings per second each; steady; each confirmed by covering it; not yet checked against a tape
Camera and time-of-flight fusion	29 offline tests pass; first live fused hazard: camera estimate 0.874 m, time-of-flight 0.096 m, reported 0.096 m (before calibration)
YOLO11n, COCO, NCNN, 320 input	7.9 frames/s, 175 ms median readout to result
Pothole model, run 1	mAP@50 59.7% on its own test set; 0.000 on South African frames
Pothole model, run 2 (road-band tiles)	mAP@50 0.714 at 640 and 0.566 at 320, on validation tiles

The time-of-flight stand-ins answered and each responded to a fingertip in the right order (centre 267 to 300 mm and right 24 to 38 mm on the bench; left 8190, the code for no target), so the first part of R8 is met; their accuracy against a tape and the fusion validation (test C9) have not been run, so the rest of

Table 4.3: Camera detector latency on the Raspberry Pi 4 (ms, median / 95th percentile; 60 s per run).

Run	fps	Queue	Pre	Inference	Alert logic	Total
4 threads	19.5	52 / 71	12 / 14	36 / 39	0.1 / 0.2	102 / 120
4 threads, live view	17.3	57 / 78	12 / 12	36 / 39	0.1 / 0.2	105 / 127
1 thread	7.0	47 / 68	12 / 14	126 / 129	0.1 / 0.2	186 / 207

R8 is outstanding. The first live fused hazard shows the matching working and how far the uncalibrated camera distance can be wrong at close range. The fusion's sensor directions and the camera's focal length are still defaults.

[Outstanding: fusion validation C9 (distance error against a tape at 0.5, 1.0 and 1.5 m), focal-length calibration, pothole model on the Raspberry Pi 4 and its held-out test score, and detection on the vehicle outdoors.]

4.7 Telemetry, logging and recovery (R9, R10)

The unit connected to the broker at the first attempt and published 47 times with no failures; a requested reconnection completed in 0.4 s. With the modem's radio switched off for 60 s, the firmware detected the loss from the modem's report 0.8 s after the radio went off, or after three failed publishes at 20.7 s, and retried after 30 and then 60 s. In v0.5.0 it reconnected 36 s after the radio returned, waiting for its next retry although data were back within 2.4 s; v0.5.3 reconnected about 5 s after. Logging continued throughout. A full read-back recovered all 6,020 stored records (2.2 MB, about 16.7 hours at one record per 10 s) after four passes; 5,545 were not valid JSON because of `nan`, and after the v0.6.3 substitution all 6,020 parse. R9 is met on the bench for the path to a test broker.

The dashboard line (v0.7.0) delivered 40 lines in 40 s with no gap in the sequence numbers, every line valid JSON and at most 272 bytes, with both IMUs at 200 Hz and no drops. Through the bridge on the Raspberry Pi 4 the record reached a laptop 0.37 s old with no malformed lines, and opening the port did not reset the telemetry unit; the pre-test check on 4 October found the record 0.5 s old and the telemetry answering in 18 ms. R10 is met on the bench.

[Outstanding: database synchronisation: completeness, delay, duplicates and backlog replay after an induced outage (firmware v0.7.4), and the same measures on the moving vehicle.]

4.8 Driver display (R11, R12)

Against the live detector, the dashboard's camera page showed video 0.6 s after it was opened and decoded 14.2 frames/s against the detector's 14.8, with no gap longer than 0.22 s; with a person in front of the camera it listed a red "Person, Immediate" alert 0.5 s after opening, and it switched to its offline state within a second of the detector stopping. In live-only mode, with the bridge down, every page showed "--" or "Unavailable" and never a simulated value, and the dashboard returned to the no-data state, not to simulation, when the bridge stopped. Run against the live bridge, the Sensors page showed the ahead sensor at 0.43 m in red, the right at 0.69 m in amber and the left with no reading, and the banner appeared over the driving page. Reading the touch controller during 25 taps, the panel reported contact together

with the first position in every tap, so it adds no measurable delay; the taps lasted 200 to 640 ms and the buttons act on release, and the processor was idle (load average 0.04). R11 is met on the bench. R12, the glance demand, has not been measured, and the layout has not been checked against the legibility figures of Section 2.6.

[Outstanding: alert onset to screen on the Raspberry Pi 5, legibility in daylight, and glance observation on the vehicle.]

4.9 Integration and power

All telemetry-unit subsystems ran together on the target board, and the chain from the telemetry unit through the Raspberry Pi 4 to the dashboard on the Raspberry Pi 5 ran on the bench. The power-flag log found the cause of the under-voltage seen throughout the bench work: under-voltage began 14 s after the telemetry unit was plugged into the Raspberry Pi 4's USB port, when its modem started, throttling the processor to 600 MHz and slowing the detector from 14.3 to 12 frames/s; on 4 October, under-voltage had held the detector at 6.1 frames/s. The Raspberry Pi 4 reached 81 °C without a fan and 48 °C with one (Appendix C).

[Outstanding: full-system test on the vehicle over a defined route, with the telemetry unit on its own supply.]

4.10 Summary against the requirements

Table 4.4: Status of each requirement at the time of writing (bench unless stated).

ID	Result	Status
R1	200.0 to 200.5 Hz, 0 dropped; gaps over 10 ms only at a flush	Met on bench; vehicle outstanding
R2	9.80 to 9.81 m/s^2 after calibration; drifts with orientation	Met at calibration; mounted recalibration outstanding
R3	Self-tests pass; 149 of 149 windows fused	Met
R4	Median 2.7 m, 95% within 11.0 m of the phone	Partly met
R5	14 valid records; link intermittent; no meter check	Partly met
R6	Median interval 500.06 ms, range 499.72 to 500.41 ms	Met
R7	19.5 frames/s; onset about 0.2 s	Met (person class)
R8	Sensors answer; no tape check; fusion not validated	Partly met
R9	47 of 47 publishes; reconnection about 5 s after coverage; full read-back	Met to a test broker; database outstanding
R10	40 of 40 lines; record 0.37 s old at the laptop	Met
R11	Live-only presentation; alerts in 0.5 s; offline state	Met
R12	Not measured	Outstanding
R13	Removable mounting planned; not yet installed	Outstanding

Chapter 5

Discussion

This chapter interprets the results against the literature and the research questions. Its main limitation is stated throughout: the results are from the bench.

5.1 Acquisition integrity and what it costs

The literature review found that edge-telemetry studies are usually judged by accuracy or mean latency, without jitter, queue behaviour or outage recovery. The results here report those quantities directly: zero dropped samples, the worst gap with and without logging, and the cause of the only recurring gap. The flash-write pause shows why they matter. Buffering cut its rate sixfold but not its length, because the pause comes from the flash itself. A 40 ms gap in a 200 Hz stream is eight missing sample instants once a minute, small for 1 s ride windows but significant for impact detection. Logging to separate storage, or on the Raspberry Pi, would remove it rather than reduce it. The open sensor settings belong to the same requirement: the 260 Hz filter lets content above 100 Hz alias into the band (v0.7.4 moves to 94 Hz), and the $\pm 8\ g$ range was probably reached by a hand disturbance, so clipped samples should be flagged before vehicle data are interpreted.

5.2 Ride characterisation (RQ1, RQ4)

The calibration result supports the review's warning that low-cost inertial measurements depend on the individual sensor and its mounting: two nominally identical units disagreed by about 8%, and their factors changed between boards because one magnitude factor holds only for one orientation. A ride index compared across sessions therefore has to be calibrated in its final mounting, which the firmware supports with one command. The features are computed correctly, but on the magnitude rather than a vehicle-fixed vertical axis and without ISO 2631 weighting, so they describe overall vibration exposure, not comfort. The equal-weight fusion and the disagreement figure are designed to show when the two mounting points see different motion; a D of 1.68 in the bench session shows that this happens even on a hand-held board, and it will be the first thing to examine on the vehicle.

RQ4 cannot be answered from the bench. The method is in place: with the motor's Hall pulses giving its speed, vibration that follows motor speed with the vehicle on stands can be separated from vibration that appears only on the road, as the review recommends [17]. It depends on the Hall tap and on the first vehicle sessions.

[Outstanding: interpretation of the road-test data: repeatability of σ and R over repeated traversals, speed dependence, and whether powertrain components are separable (to be written after 14 October 2026).]

5.3 Hazard sensing

The camera pipeline is fast enough for its purpose: an alert about 0.2 s after an object appears is about 1.7 m of travel at 30 km/h. The weak point is distance, not speed. The first fused hazard put a person at 0.874 m by the camera's width heuristic and 0.096 m by the time-of-flight sensor, so a camera-only band can be a whole band wrong at close range; this is the case for fusion, in which the camera says what and where and the sensor says how far. The stand-in sensors' 1.2 m range is only about 0.14 s of travel at 30 km/h, so they serve low-speed manoeuvring, and the camera remains the only source at road speed until longer-range sensing is fitted. The pothole models show the same dependence on local data that the review noted for roughness classifiers: a model scoring 59.7% on its own source found almost none of the potholes in South African footage, and training on the road band of that footage raised the validation score to 0.714. The held-out score and a run on the vehicle's own camera, which has no infrared-cut filter, are needed before that improvement can be claimed.

5.4 Data integrity and connectivity (RQ3)

Logging independently of the connection, and reconnecting when the modem reports that data are available, gave the store-and-forward behaviour the review identified as necessary for intermittent coverage. The read-back is the important result: it recovered every record and found a fault, invalid JSON for missing values, that watching the live stream had not shown. The cellular path is not complete, however: data reach a public test broker, not a database, and the backlog is not yet replayed after an outage, so the review's measures of completeness, delay and duplicates cannot yet be reported.

[Outstanding: database synchronisation results and their interpretation against the store-and-forward measures of Chapter 2.]

5.5 Driver display (RQ2)

The choice of the existing dashboard over a windshield HUD is supported by the review: a head-up position does not make a display safe by itself, its optics are hard to make legible on a removable unit in daylight, and a second display adds a competing channel. Of the five dashboard issues, data validity and update age were the ones that integration showed to matter most: the inherited dashboard showed simulated speed and battery values without any sign whenever its data link was down, which is worse than

showing nothing, and live-only mode with a 3 s age limit removes that. Layout and alert priority follow the review's guidance on few elements and a stable hierarchy, with only immediate hazards interrupting the driving page. Contrast is the weakest of the five, because it rests on the team's colours and has not been measured in daylight. The touch measurement shows that the panel is not a source of delay, so the slowness noticed comes from the buttons acting on release, which can be changed. Whether the display can be read within the 2 s guidance has not been measured; the character-size and touch-target figures of Section 2.6 give a check that can be applied before the on-vehicle observation.

5.6 Integration and power

The most consequential finding of the integration was not in any one subsystem. The telemetry unit's LTE modem, powered from the Raspberry Pi's USB port, pulled the supply into under-voltage whenever it transmitted, throttled the processor, slowed the detector and is the likely reason the Raspberry Pi 4 stopped booting. The modem worked correctly throughout; the fault was in how the parts were powered together. A separate supply for the telemetry unit is therefore a requirement for the vehicle installation, and a measured power budget against the 72 V source, which the brief asks for, is still missing.

5.7 Validity and limitations

All results come from one set of hardware on the bench, without the vehicle's vibration, heat, supply and lighting. Several parameters are still defaults or uncalibrated: the camera's focal length, the time-of-flight directions, the accelerometer range and filter, and the GNSS speed units. The pothole score is on validation tiles. These limits do not invalidate the bench results, which measure what they claim to measure, but they bound what can be concluded from them.

Chapter 6

Conclusions

This project set out to build a low-cost edge-processing telemetry unit for the TUKZIE Rev 0 electric cargo trike and to present its results safely to the driver on the vehicle's existing dashboard. Each research question is answered here from the evidence in Chapter 4, separating what has been shown from what depends on the vehicle.

RQ1: accuracy and repeatability of the dynamic-response measurement. The measurement chain is sound on the bench: two IMUs are sampled at a fixed 200 Hz with no lost samples, including with every other subsystem running; each is calibrated against gravity, which corrects a measured 8% disagreement between nominally identical units; and the features and the dual-sensor fusion are computed correctly. How accurately and repeatably this chain characterises the trike's response has not been measured, because no data have yet been recorded on the moving vehicle. RQ1 is answered for the instrument but not for the vehicle.

RQ2: presenting safety-critical information on the existing dashboard. A sparse presentation was implemented on the vehicle's own dashboard: a camera page with the live view and active alerts, a Sensors page with distances and fused hazards, and a banner over every page for an immediate hazard, all showing only live data and marking missing or out-of-date data as absent. On the bench the alerts reach the page within half a second of opening and the touch panel adds no measurable delay. Whether a driver can read this within the 2 s glance guidance has not been measured, so RQ2 is answered for the information hierarchy and update strategy but not for glance time.

RQ3: latency, data loss and recovery. On the bench, acquisition loses no samples; the only recurring pause, from writing the log to flash, is bounded to one 33 to 40 ms gap a minute; the camera produces a result a median of 102 ms after the sensor is read; every publish succeeded; the unit reconnects about 5 s after coverage returns; and the local log is recovered completely. RQ3 is answered for the bench up to a test broker; database synchronisation and the same measures on the moving vehicle remain.

RQ4: features that separate road from powertrain vibration. The time-domain features are implemented and verified, and the method for separating the two sources, using the motor's Hall pulses

as a speed reference, is designed. Without vehicle data and the Hall tap, RQ4 is not answered.

[Outstanding: revise each answer with the road-test results of 14 October 2026.]

Overall. The project delivers an integrated, low-cost architecture for a three-wheeled electric vehicle: fixed-rate dual-sensor acquisition with measured timing and loss, on-board ride features, camera detection fused with time-of-flight ranging, cellular telemetry with local logging and automatic reconnection, and presentation on the vehicle's existing dashboard. Each part has been measured on the bench, and integrating them exposed faults that no subsystem test showed, the most important being the undervoltage caused by powering the LTE modem from the Raspberry Pi. The work has reached a working bench prototype rather than the demonstration in a relevant environment set as the target in Chapter 1; the on-vehicle sessions and the calibrations listed in Chapter 5 are what remain, and Chapter 7 sets out the work in order of priority.

Chapter 7

Recommendations

The recommendations are in order of priority, the first being those on which the validity of the vehicle data depends.

1. Acquisition integrity. Remove the flash-write pause rather than reduce it: move logging off the acquisition processor (to the Raspberry Pi or separate storage), or place the acquisition code in instruction RAM, or read the IMUs through their hardware FIFOs so that a pause is absorbed. Set the IMU filter to 94 Hz (done in v0.7.4, to be verified), or sample faster and decimate in software, and flag samples at the $\pm 8\,g$ limit so that clipped windows are marked. Set the range from the peaks recorded on the vehicle.

2. Calibration. Recalibrate both IMUs in their mounted position before every measurement campaign, and replace the single magnitude factor with a multi-position per-axis calibration so that the result no longer depends on orientation. Calibrate the camera's focal length, the fusion's sensor directions and each time-of-flight sensor against a tape (test C9), confirm the GNSS speed units against a known speed, and check the BMS voltage against a meter.

3. Power. Give the telemetry unit its own regulated supply from the vehicle, never a Raspberry Pi's USB port; give the Raspberry Pi 4 a 5.1 V, 3 A supply and cooling; and measure the current of each unit to produce the power budget against the 72 V pack that the brief asks for.

4. Synchronisation. Replace the jumper on the sync line with a locking connector; drive an LED from the sync pin within the camera's view to measure the frame-to-edge offset optically; and record the mapping of frames to sync edges in every session so that video and inertial events can be aligned after the fact.

5. Backlog recovery. Give every record a sequence number, mark records as published only on acknowledgement, replay the unsent backlog after an outage, rotate rather than truncate the log, and ingest into a database whose completeness, delay and duplicate rate are reported after an induced outage. Move from the public test broker to an authenticated, encrypted one.

6. Display failure behaviour. Extend the live-only rule to every field with its own age (for example, mark the battery state as stale when the BMS stops reporting); make the navigation buttons act on press; measure alert onset to screen on the Raspberry Pi 5 and the legibility of each page in daylight against the ISO 15008 character-size figure; and carry out a glance observation in a controlled low-risk trial.

7. Ride analysis. Wire the Hall tap, set the pole-pair count, record the stationary powertrain baseline, add a spectral feature (band energy or power spectral density) and orientation compensation, and then test which features separate road from powertrain vibration (RQ4).

8. Obstacle detection. The detector recognises only its trained classes and cannot report unexpected debris. Road-obstacle and anomaly detection is a separate research area, with synthetic-data approaches [41], dedicated benchmarks [42] and lightweight detectors trained on rare roadside obstacles [43], whose speed would have to be measured on the Raspberry Pi. Until then the time-of-flight sensors' unclassified obstacles cover close range, and the pothole model should be validated on its held-out test set and on the vehicle's camera before it raises any alert.

Bibliography

[1] United Nations, Department of Economic and Social Affairs, Population Division, *World Urbanization Prospects: The 2018 Revision*. New York, NY, USA: United Nations, 2019.

[2] Jiangsu Jinpeng Group Co., Ltd, *Electric Tricycle User Manual*. Vehicle documentation, TUKZIE Rev 0 platform.

[3] N. Adams and S. Rasara, "Jinpeng Electric Tricycle Drivetrain: Characterization & Modelling," TUKZIE vacation work report, University of Cape Town, Department of Electrical Engineering, Jul. 2026.

[4] O. S. Sebwa, P. V. Chombo, G. B. Gerutu, R. O. Kivugo, J. H. Hosea, and K. A. Greyson, "Energy, economy and environmental assessment of e-tricycle for medium-size cargo deliveries: A case of Dar es Salaam City," *The Journal of Engineering*, vol. 2025, no. 1, Art. no. e70073, 2025, doi: 10.1049/tje2.70073.

[5] World Health Organization, *Global Status Report on Road Safety 2023: Summary*. Geneva, Switzerland: World Health Organization, 2023.

[6] National Highway Traffic Safety Administration, *Visual-Manual NHTSA Driver Distraction Guidelines for In-Vehicle Electronic Devices*, Federal Register, vol. 78, no. 81, pp. 24818–24890, Apr. 2013.

[7] S. Winberg and S. Jayalath, "Development of an Embedded HPEC Telemetry Unit and Windshield HUD for the TUKZIE Rev 0 Platform," EEE4022S project brief, University of Cape Town, Department of Electrical Engineering, 2026.

[8] Y. Zhu, Y. Jing, M. Jiang, Z. Zhang, D. Wang, and W. Liu, "An experimental study of the cognitive load of in-vehicle multiscreen connected HUD," in *Design, User Experience, and Usability: Design for Contemporary Technological Environments*, Lecture Notes in Computer Science, vol. 12781. Cham, Switzerland: Springer, 2021, pp. 268–285, doi: 10.1007/978-3-030-78227-6_20.

[9] W. M. Kimmel, C. D. Weiss, and A. M. Mazzuchi, *Technology Readiness Assessment Best Practices Guide*, NASA/SP-20205003605, 2020.

[10] J. Saenz, M. Figliozzi, and J. Faulin, "Assessment of the carbon footprint reductions of tricycle logistics services," *Transportation Research Record*, vol. 2570, no. 1, pp. 48–56, 2016, doi: 10.3141/2570-06.

[11] Federal Highway Administration, *Measuring and Specifying Pavement Smoothness*, Tech. Brief FHWA-HIF-16-032, Washington, DC, USA, 2016.

[12] A. González, E. J. O'Brien, Y.-Y. Li, and K. Cashell, "The use of vehicle acceleration measurements to estimate road roughness," *Vehicle System Dynamics*, vol. 46, no. 6, pp. 483–499, 2008, doi: 10.1080/00423110701485050.

[13] S. Sattar, S. Li, and M. Chapman, "Road surface monitoring using smartphone sensors: A review," *Sensors*, vol. 18, no. 11, Art. no. 3845, 2018, doi: 10.3390/s18113845.

[14] Y. I. Alatoom and O. Smadi, "Embedded framework for low-cost pavement condition evaluation using microcontroller and single-board computer platforms," *Automation in Construction*, vol. 178, Art. no. 106442, 2025, doi: 10.1016/j.autcon.2025.106442.

[15] V.-H. Bui, D.-N. Tran, P. Q. Huy, N. V. Cong, and T.-H. Dao, "Signal processing-driven real-time rough road detection with GPS accuracy improvement on edge microcontrollers," in *Proc. IEEE 15th Int. Conf. Communication Systems and Network Technologies*, 2026, doi: 10.1109/CSNT69054.2026.11502079.

[16] B. Yiliguoqi, B. Siriguleng, Arong, G. Guo, and J. Wang, "Research on the evaluation and analysis of road surface roughness based on smartphone sensors and SVM," *Scientific Reports*, vol. 16, Art. no. 4409, 2026, doi: 10.1038/s41598-025-34396-3.

[17] Z. Li, W. Yu, and X. Cui, "Online Classification of Road Roughness Conditions with Vehicle Unsprung Mass Acceleration by Sliding Time Window," *Shock and Vibration*, vol. 2018, Art. no. 5131434, 2018, doi: 10.1155/2018/5131434.

[18] International Organization for Standardization, *Mechanical Vibration and Shock: Evaluation of Human Exposure to Whole-Body Vibration, Part 1: General Requirements*, ISO 2631-1:1997, including Amendment 1:2010.

[19] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, "Edge computing: Vision and challenges," *IEEE Internet of Things Journal*, vol. 3, no. 5, pp. 637–646, 2016, doi: 10.1109/JIOT.2016.2579198.

[20] M. Barnell, C. Raymond, C. Capraro, D. Isereau, C. Cicotta, and N. Stokes, "High-performance computing (HPC) and machine learning demonstrated in flight using Agile Condor," in *Proc. IEEE High Performance Extreme Computing Conf.*, 2018, pp. 1–4, doi: 10.1109/HPEC.2018.8547797.

[21] A. Pérez-González, A. F. Villa-Salazar, I. N. Gomez-Miranda, J. D. Velásquez-Gómez, A. F. Romero-Maya, and A. Jaramillo-Duque, "An IoV-based real-time telemetry and monitoring system for electric racing vehicles: Design, implementation, and field validation," *Vehicles*, vol. 7, no. 4, Art. no. 128, 2025, doi: 10.3390/vehicles7040128.

[22] Y.-C. Liu, "Effects of using head-up display in automobile context on attention demand and driving performance," *Displays*, vol. 24, nos. 4–5, pp. 157–165, 2003, doi: 10.1016/j.displa.2004.01.001.

[23] M. Smith, J. L. Gabbard, and C. Conley, "Head-up vs. head-down displays: Examining traditional methods of display assessment while driving," in *Proc. 8th Int. Conf. Automotive User Interfaces and Interactive Vehicular Applications*, 2016, pp. 185–192, doi: 10.1145/3003715.3005419.

[24] A. Mareška, T. Kordová, and M. Havlík Míka, "Production of Head-Up Display windshield and its relation with the image quality," *Transactions on Transport Sciences*, vol. 13, no. 2, pp. 63–70, 2022, doi: 10.5507/tots.2022.011.

[25] Z. Qin, F.-C. Lin, Y.-P. Huang, and H.-P. D. Shieh, "Maximal acceptable ghost images for designing a legible windshield-type vehicle Head-Up Display," *IEEE Photonics Journal*, vol. 9, no. 6, pp. 1–12, 2017, doi: 10.1109/JPHOT.2017.2758820.

[26] R. Häuslschmid, B. Pfleging, F. Alt, and A. Butz, "A design space to support the development of windshield applications for the car," in *Proc. 2016 CHI Conf. Human Factors in Computing Systems*, 2016, pp. 5076–5091, doi: 10.1145/2858036.2858336.

[27] R. Häuslschmid, S. Osterwald, M. Lang, and A. Butz, "Augmenting the driver's view with peripheral information on a windshield display," in *Proc. 20th Int. Conf. Intelligent User Interfaces*, 2015, doi: 10.1145/2678025.2701393.

[28] K. Park and Y. Im, "Ergonomic guidelines of Head-Up Display user interface during semi-automated driving," *Electronics*, vol. 9, no. 4, Art. no. 611, 2020, doi: 10.3390/electronics9040611.

[29] B. Reimer, B. Mehler, J. Dobres, J. F. Coughlin, S. Matteson, D. Gould, N. Chahine, and V. Levantovsky, "Assessing the impact of typeface design in a text-rich automotive user interface," *Ergonomics*, vol. 57, no. 11, pp. 1643–1658, 2014, doi: 10.1080/00140139.2014.940000.

[30] F. You, Y.-F. Yang, M.-T. Fu, J. Zhang, and J.-M. Wang, "Design guidelines for the size and length of Chinese characters displayed in the intelligent vehicle's central console interface," *Information*, vol. 12, no. 5, p. 213, 2021, doi: 10.3390/info12050213.

[31] H. Kim, S. Kwon, J. Heo, H. Lee, and M. K. Chung, "The effect of touch-key size on the usability of in-vehicle information systems and driving safety during simulated driving," *Applied Ergonomics*, vol. 45, no. 3, pp. 379–388, 2014, doi: 10.1016/j.apergo.2013.05.006.

[32] Euro NCAP, *Safe Driving: Driver Engagement Protocol*, version 1.2. Leuven, Belgium: Euro NCAP, 2026.

[33] D. L. Strayer, J. M. Cooper, R. M. Goethe, M. M. McCarty, D. J. Getty, and F. Biondi, *Visual and Cognitive Demands of Using In-Vehicle Infotainment Systems*. Washington, DC, USA: AAA Foundation for Traffic Safety, 2017.

[34] M. De Simone, Z. B. Rivera Chavez, and D. Guida, "Obstacle Avoidance System for Unmanned Ground Vehicles by Using Ultrasonic Sensors," *Machines*, vol. 6, no. 2, Art. no. 18, 2018, doi: 10.3390/machines6020018.

[35] N. Noomwongs, K. Kiataramgul, S. Chantranuwathana, and G. Phanomchoeng, "GNSS-RTK-Based Navigation with Real-Time Obstacle Avoidance for Low-Speed Micro Electric Vehicles," *Machines*, vol. 13, no. 6, Art. no. 471, 2025, doi: 10.3390/machines13060471.

[36] G. Caroleo, A. Albini, and P. Maiolino, "On the Characterisation of the Time-of-Flight VL53L5CX Sensor by STMicroelectronics for Indoor Robotics Applications," *Sensors*, vol. 26, no. 5, Art. no. 1639, 2026, doi: 10.3390/s26051639.

[37] Makerfabs, "ESP32S3 4G LTE CAT1 A7670X," product page and hardware repository. [Online]. Available: `https://www.makerfabs.com/esp32s3-4g-lte.html` and `https://github.com/Makerfabs/ESP32-S3-4G-LTE-CAT1-A7670X` (accessed 24 Sep. 2026).

[38] SIMCom Wireless Solutions, *A76XX Series AT Command Manual*, V1.09, Apr. 2023. [Online]. Available: `https://files.waveshare.com/wiki/A7670E-Cat-1-GNSS-HAT/A76XX_Series_AT_Command_Manual_V1.09.pdf` (accessed 24 Sep. 2026).

[39] Sensirion AG, *Datasheet SEN5x: Environmental Sensor Node for HVAC and Air Quality Applications*, version 2, Stäfa, Switzerland, Mar. 2022.

[40] Raspberry Pi Ltd, *The Picamera2 Library*, Sep. 2026. [Online]. Available: `https://datasheets.raspberrypi.com/camera/picamera2-manual.pdf` (accessed 28 Sep. 2026).

[41] T. E. Choe, J. Wu, X. Lin, K. Kwon, and M. Park, "HazardNet: Road Debris Detection by Augmentation of Synthetic Models," arXiv:2303.07547, 2023.

[42] R. Chan, K. Lis, S. Uhlemeyer, H. Blum, S. Honari, R. Siegwart, P. Fua, M. Salzmann, and M. Rottmann, "SegmentMeIfYouCan: A Benchmark for Anomaly Segmentation," in *Proc. Neural Information Processing Systems (NeurIPS) Track on Datasets and Benchmarks*, 2021.

[43] A. Bint-e Tanveer, M. A. Kamal, M. M. Alam, and M. Mohd Su'ud, "Real-time detection of rare roadside obstacles using YOLOv8-n in autonomous vehicles," *PLOS ONE*, 2026, doi: 10.1371/journal.pone.0350732.

[44] Raspberry Pi Ltd, "Raspberry Pi hardware: GPIO and the 40-pin header," Raspberry Pi documentation. [Online]. Available: https://www.raspberrypi.com/documentation/computers/raspberry-pi.html

Appendix A

Graduate Attributes, AI Use and Project Repository

A.1 Graduate attribute evidence

This section summarises how the work reported here meets the six graduate attributes assessed in this course, with the sections that hold the evidence.

GA1: Problem solving

The research questions were framed around measurable vehicle dynamic response rather than subjective ride comfort, and each was tied to a requirement and a verification method (Section 1.4). The design choices were reasoned from first principles: a 200 Hz sampling rate from the ISO 2631-1 band and the sampling theorem, drift-free task timing and non-blocking bounded queues so that acquisition is never held up (Section 3.3), the acceleration magnitude as an orientation-tolerant signal (Section 3.4), and speed normalisation after the literature showed that raw RMS mixes road roughness with speed (Section 3.4.2). Faults were root-caused from evidence rather than worked around, including a sync-pin choice checked against the board schematic (Section 3.8.1), a serial-routing fault (Section 3.10.2) and the flash-write acquisition stall (Section 3.9).

GA4: Investigations, experiments and data analysis

Assumptions were treated as hypotheses and tested on the real hardware: the GNSS reply format (assumed 13 fields, measured 9, Section 3.6), the BMS service layout (Section 3.5), and the cause of the periodic acquisition stall, confirmed by a controlled comparison with local logging on and off (Section 3.9). The feature computation was validated against a synthetic signal with known answers, the two IMUs were calibrated against gravity and checked under disturbance (Section 3.4.1), the dual-IMU fusion was verified in a recorded session (Section 3.4.3), and the camera-to-IMU timing offset was bounded (Section 3.8.1). The raw evidence is indexed in Appendix D.

GA5: Use of engineering tools

PlatformIO, the Arduino framework and FreeRTOS were used for the firmware, with esptool and serial diagnostics for hardware verification (Appendix E). Python tools were written to support testing: a scripted bench test, a browser bench console that parses and replays the telemetry, and schematic checks. On the camera side, LiteRT and Picamera2 run the detector, and per-stage latency is measured from the sensor's own frame timestamps (Section 3.8.2). A parametric bracket was modelled in OpenSCAD and a camera casing was designed and 3D printed (Section 3.2). The limits of each tool were checked rather than assumed, for example a BMS checksum formula that no vendor document confirmed and that proved wrong on the vehicle (Section 3.5).

GA6: Professional and technical communication

This report documents the engineering work, including the faults found and how they were resolved. The written work was developed with the supervisor through regular meetings and a draft Introduction and Methodology, and inconsistencies between chapters were corrected as the work changed. The raw evidence behind each result is kept and indexed (Appendix D), and the firmware, tools and planning documents are in the project repository.

GA8: Individual work

The hardware bring-up, firmware, bench tools and report were done individually, with the supervisor consulted on specific questions. The components were selected, compared against the requirements and ordered by the author, for example the 8×8-zone time-of-flight sensor chosen over a single-point one to separate left, centre and right (Section 3.8.3); each component was tested on the bench before being relied on; and the camera casing was designed and 3D printed by the author (Section 3.2). Inherited work is identified where it is used: the vehicle dashboard and its team, and a teammate's BMS parser used only as an independent cross-check (Section 3.5). The use of AI tools is declared in Section A.3.

GA9: Independent learning ability

The work required learning FreeRTOS timing, BLE GATT, the A76XX AT command set and on-device inference on the Raspberry Pi, largely from primary sources: manufacturer schematics and manuals, and live tests on the hardware. Assumptions were repeatedly corrected against those sources, for example GPIO assignments against the schematic and the MQTT result handling against the modem manual (Section 3.9). A continuously running literature search was kept up so that the research-gap claims were tested rather than asserted once, and it narrowed two of them (Chapter 2).

A.2 Project repository

Firmware, camera software, planning documents and supporting tools for this project are available at `https://github.com/0geder/Tukzie-HPEC-HUD`.

A.3 Declaration of Artificial Intelligence Use

Artificial intelligence tools were used to assist with clarifying the organisation of this report, improving the alignment and consistency of its presentation, refining wording, and checking LaTeX formatting. All AI-assisted suggestions were reviewed by the author. The author remains responsible for the report's technical content, analysis, interpretations, and final editorial decisions.

Appendix B

Supporting Figures

The figures below support the text of Chapters 1 and 3, where each is referenced. The parametric bracket of Figure B.4 was modelled first but not printed; the printed casing of Figure B.5 was used instead.



Figure B.1: Manufacturer identification plate on the TUKZIE Rev 0 platform, photographed directly during vehicle inspection.

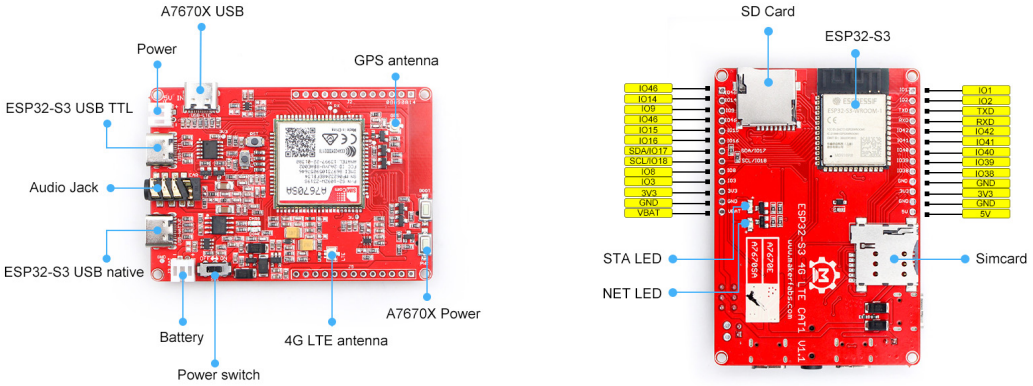


Figure B.2: Makerfabs ESP32-S3 4G LTE CAT1 A7670X board: manufacturer-labelled ports (left) and pinout (right) [37].

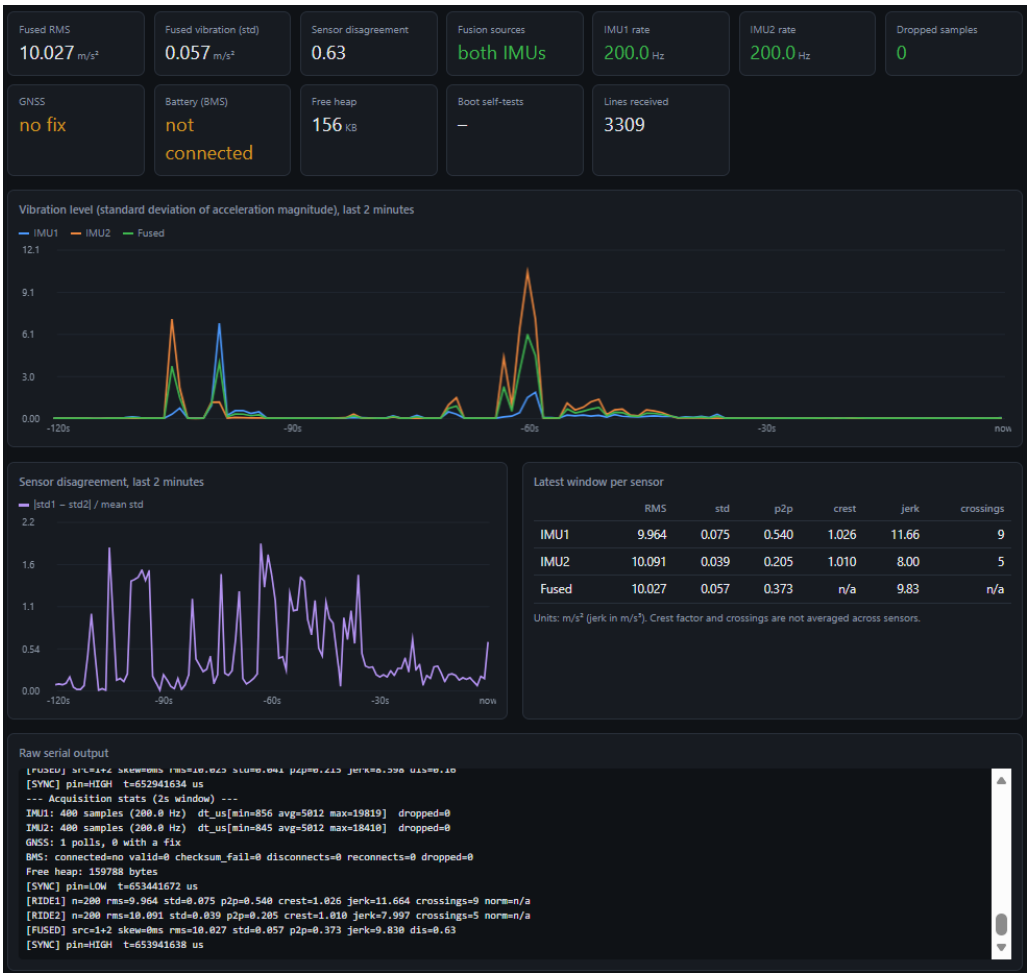


Figure B.3: The live telemetry dashboard connected to the telemetry unit over USB during bench testing, showing both IMUs fused at 200 Hz with no dropped samples and two hand disturbances in the vibration plot.

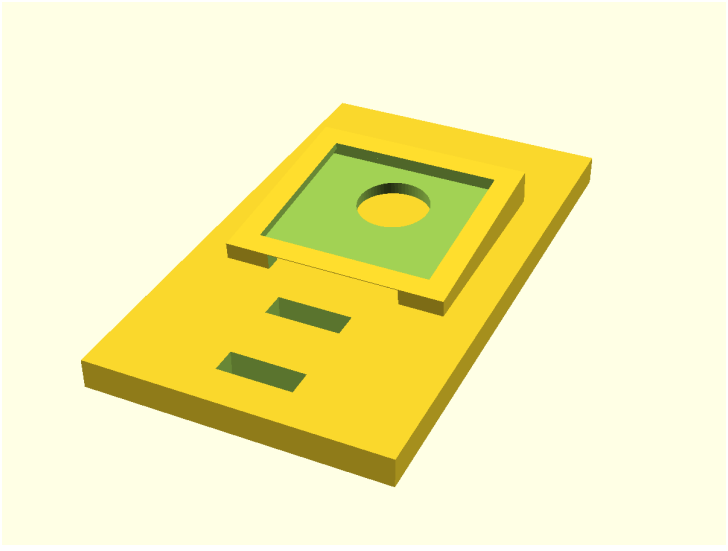


Figure B.4: Initial parametric camera bracket (OpenSCAD, full render), showing the corrected lens-retention lip and cable-routing slots. This design was not printed.

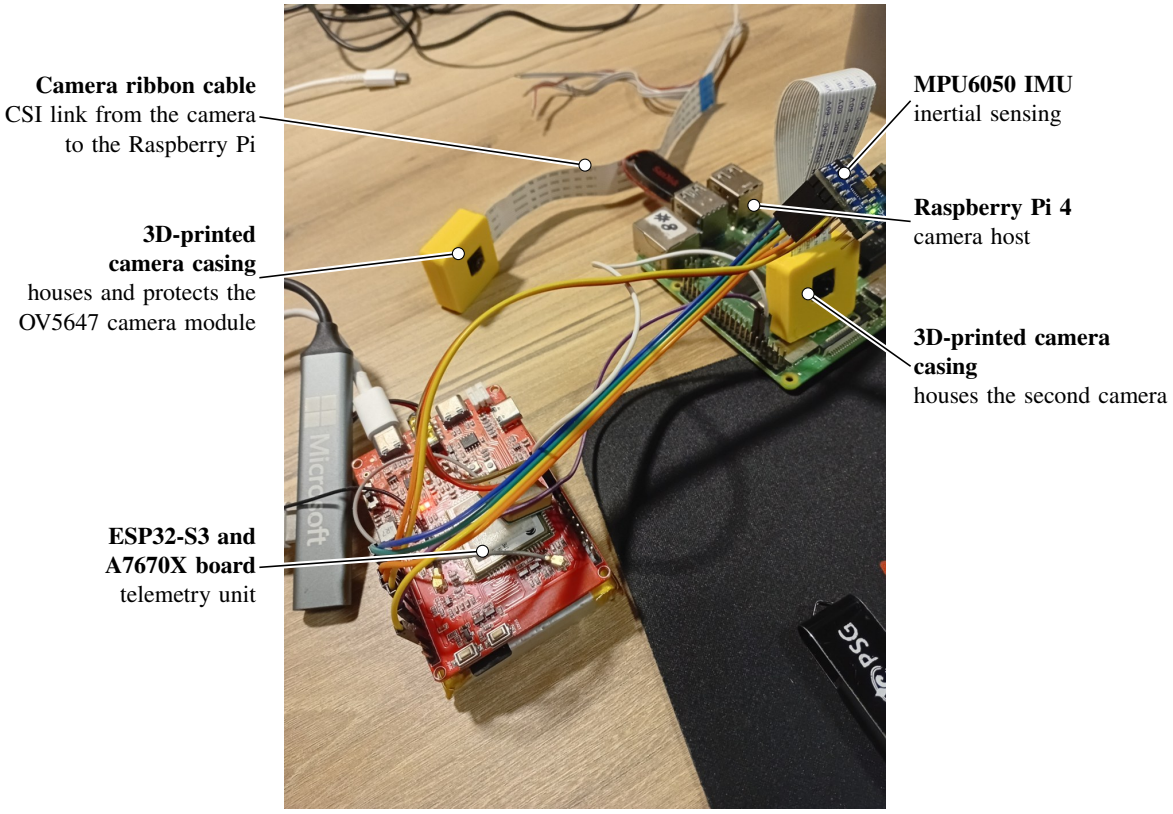


Figure B.5: The cameras fitted in their 3D-printed casings, shown with the rest of the bench setup.

Appendix C

Wiring and Connections

This appendix records how every part of the system is connected, so that it can be rebuilt or checked from this document alone. Each table gives both ends of each wire by connector, pin number and signal. The pin assignments come from the firmware (`TelemetryUnit/src/main.cpp`), the time-of-flight reader (`CameraDetection/tof_reader.py`) and the bench procedures (`BenchTest/TEST_PROCEDURES.md`), and the pin positions from the manufacturers' documents reproduced in the figures. Connections not yet made on the hardware are marked as planned.

C.1 Pin references

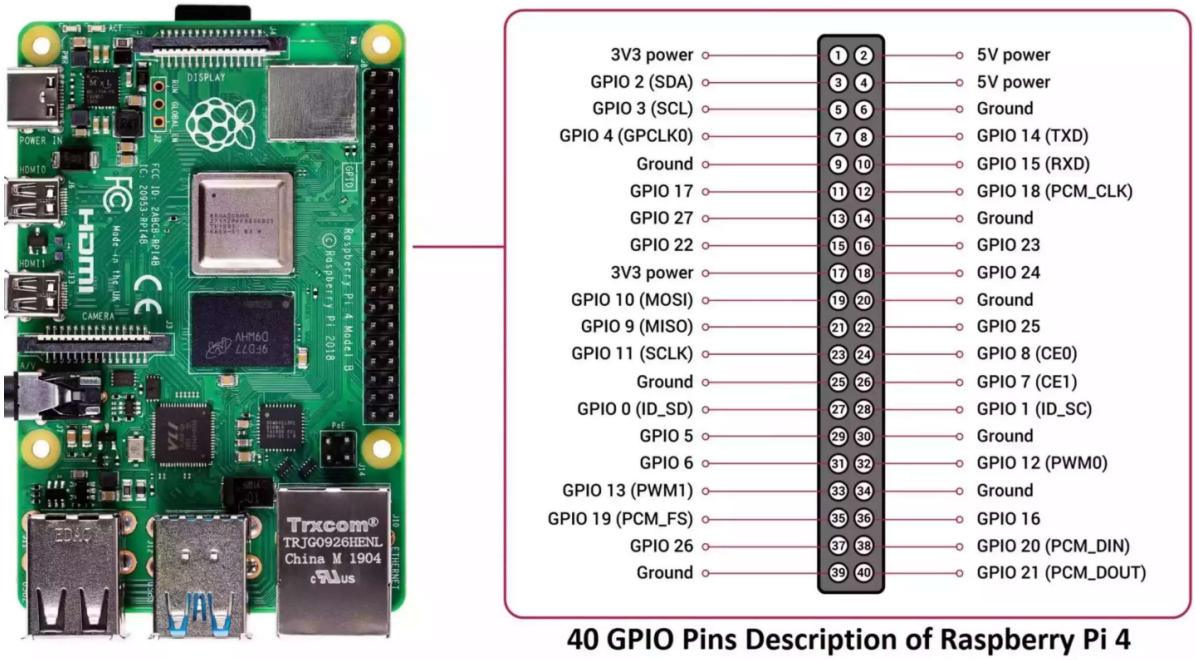


Figure C.1: Raspberry Pi 4 40-pin header, physical pin numbers and functions, from the project's reference sheet (`References/Hardware/Rasp Pi 4 pins.pdf`) [44]. Odd-numbered pins are on the inner row and even-numbered pins on the board edge.

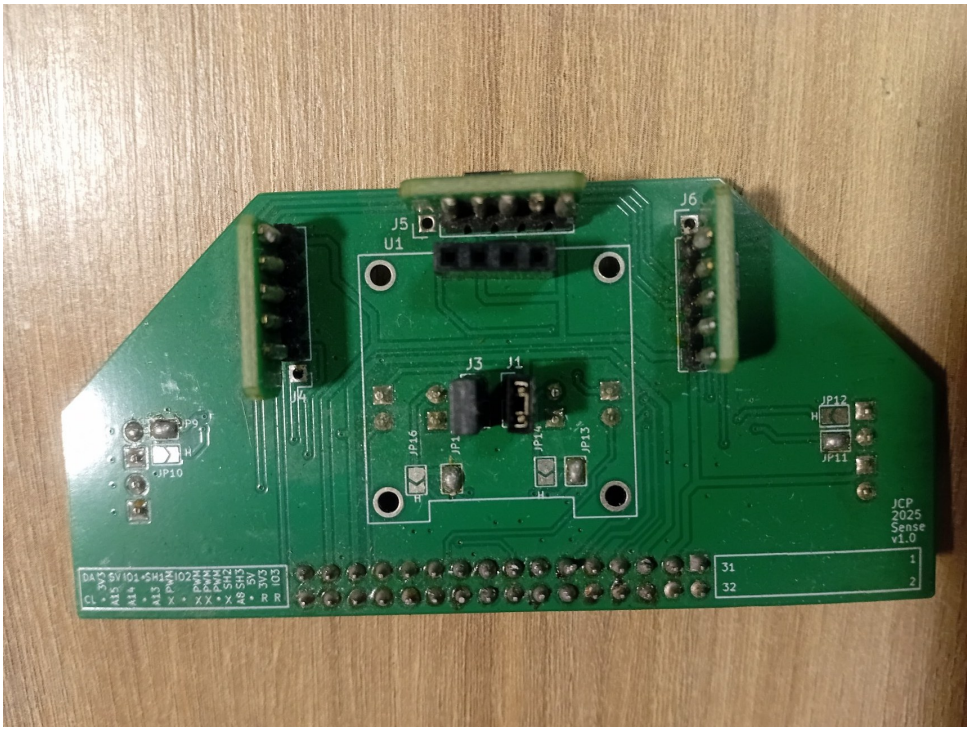


Figure C.2: Sensor board (UCT micromouse "JCP 2025 Sense v1.0") used as the time-of-flight stand-in. The 2×16 header J2 runs along the top edge; the silkscreen marks pins 1 and 2 at one end and 31 and 32 at the other, and the legend at the right names the signals (3V3, DA, CL, SH1 to SH3, 5V). The three VL53L0X sensor carriers sit on J4, J5 and J6, facing left, ahead and right.

Part	Connector
Sensor side	ACES 51451-0060N-001
Cable side	Compatible with ACES 51452-006H0H0-001 (e.g., JST GHR-06V-S)

Table 10: SEN5x connector

Pin	Name	Description	Comments
1	VDD	Supply voltage	5V ± 10%
2	GND	Ground	
3	SDA	Serial data input / output	LVTTL 3.3V compatible
4	SCL	Serial clock input	LVTTL 3.3V compatible
5	SEL	Interface select	Connect to GND
6	NC	Do not connect	

Table 11: SEN5x pin assignment.

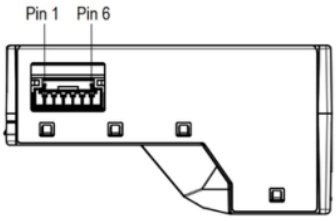


Figure 4 Pin layout. The communication interface connector (ACES 51451-0060N-001) is located at the side of the sensor adjacent to the air outlet.

Figure C.3: Sensirion SEN5x interface connector and pin assignment, reproduced from the datasheet [39]. The mating plug is a JST GHR-06V-S (1.25 mm pitch, six way).

The Makerfabs ESP32-S3 A7670X board's headers are shown in Figure B.2 (Appendix B). In this appendix its left header is called J5 and its right header J2, as on the board's schematic, with pins numbered from the top of the pinout figure: J5 runs IO46, IO14, IO9, IO46, IO15, IO16, IO17, IO18, IO8, IO3, 3V3, GND, VBAT (pins 1 to 13). The firmware comments confirm pins 5, 7 and 8 against the schematic; the others are read from the figure.

C.2 Raspberry Pi 4 to the time-of-flight sensor board

The three VL53L0X sensors share one I^2C bus and all start at address 0x29. Each sensor's shutdown line (XSHUT) goes to its own Pi GPIO, so that the reader can hold all three off, wake one at a time and move each to its own address (0x30, 0x31, 0x32). The sensors have 10 kΩ pull-ups on XSHUT and no I^2C pull-ups; the Pi's own 1.8 kΩ pull-ups on pins 3 and 5 are used.

Table C.1: Raspberry Pi 4 to sensor board J2 (Figures C.1 and C.2).

Signal	**Sensor board J2**	**Pi 4 pin**	**Pi 4 function**
3.3 V supply	pin 3 (3V3)	1	3V3 power
Ground	pin 23 (GND)	6 (or 14)	Ground
I^2C data	pin 31 (DA)	3	GPIO2, SDA
I^2C clock	pin 32 (CL)	5	GPIO3, SCL
XSHUT, left sensor	pin 21 (SH1)	15	GPIO22
XSHUT, ahead sensor	pin 9 (SH2)	16	GPIO23
XSHUT, right sensor	pin 7 (SH3)	18	GPIO24
LED supply (VDD)	not connected		

If a cooling fan uses Pi pins 4 and 6, the sensor ground goes to pin 14. The sensors are 3.3 V parts and are never connected to the 5 V pins (2 and 4). If one XSHUT wire is disconnected, that sensor's pull-up keeps it on at 0x29, where it collides with each sensor being woken, and none of the three starts; the reader then logs each sensor as not answering after a 0.5 s timeout.

C.3 Telemetry unit (ESP32-S3) to the Raspberry Pi 4

Table C.2: ESP32-S3 board to Raspberry Pi 4.

Link	**ESP32-S3 board**	**Raspberry Pi 4**
Data and 5 V (telemetry lines, `/dev/ttyACM0`)	"ESP32-S3 USB native" USB-C port	any USB port, through a powered USB hub
Time-sync pulse (3.3 V logic)	J5 pin 5, GPIO15	pin 11, GPIO17
Ground for the pulse	J5 pin 12, GND	pin 9, Ground

The powered hub is required: with the board powered from the Pi's own USB port, the LTE modem's transmit current caused repeated under-voltage on the Raspberry Pi 4 (Section 3.8.4 and Chapter 3). The two boards' 3.3 V logic levels match, so the pulse wire connects directly.

C.4 Telemetry unit sensors and antennas

Table C.3: ESP32-S3 board to the two MPU6050 inertial sensors. Each sensor has its own I^2C bus, so both use address 0x68 (AD0 left low).

MPU6050 pin	**IMU1 (bus 0)**	**IMU2 (bus 1)**
VCC	J5 pin 11, 3V3	J5 pin 11, 3V3
GND	J5 pin 12, GND	J5 pin 12, GND
SDA	J5 pin 9, GPIO8	J5 pin 7, GPIO17
SCL	J5 pin 3, GPIO9	J5 pin 8, GPIO18
AD0, INT, XDA, XCL	not connected	not connected

Table C.4: Other telemetry unit connections.

Part	**Connection on the ESP32-S3 board**	**Notes**
LTE antenna	“4G LTE antenna” u.FL socket	away from metal and from the GNSS antenna
GNSS antenna	“GPS antenna” u.FL socket	ceramic face up, clear view of the sky
SIM card	SIM holder on the underside	
A7670X modem	on board: UART RX GPIO47, TX GPIO48, PWRKEY GPIO4, RESET GPIO5	no external wiring
Battery management system	Bluetooth Low Energy, no wire	board within about 1 m of the pack

Table C.5: Sensirion SEN55 to the ESP32-S3 board (planned, not yet connected or tested; Figure C.3). The sensor shares IMU2’s bus; its address, 0x69, does not clash with the MPU6050 at 0x68.

SEN55 pin	**Signal**	**ESP32-S3 board**
1	VDD, 5 V ±10%	J2 pin 13, 5V
2	GND	J2 pin 12, GND
3	SDA (3.3 V compatible)	J5 pin 7, GPIO17
4	SCL (3.3 V compatible)	J5 pin 8, GPIO18
5	SEL	GND (selects I^2C)
6	NC	not connected

C.5 Camera, displays and power

Table C.6: Remaining connections.

Part	Connection	Notes
Camera (OV5647 NoIR)	15 cm ribbon to the Pi 4 "CAMERA" connector	contacts face the HDMI ports; longer ribbons pick up noise
Raspberry Pi 4 power	USB-C, 5.1 V 3 A supply	a weaker supply or a long thin cable gives under-voltage
Raspberry Pi 4 cooling	fan on 5 V and GND	needed: 81 °C without it, 48 °C with it (10 Oct 2026)
Raspberry Pi 5 (dashboard)	the vehicle's existing dashboard computer and display, unchanged	reaches the Pi 4 over Wi-Fi, found by its beacon
Network	both Raspberry Pis on one Wi-Fi network (laptop or phone hotspot on 2.4 GHz)	

C.6 Wiring checks

Before power is applied: every board is switched off and unplugged; each wire is checked end to end with a multimeter on continuity; 3.3 V and ground are checked not to be shorted; and each bundle is tied and fixed close to the board, so that a pull on the cable is taken by the tie and not by the header pins, since loose jumpers came off twice during bench testing. After power-up, the pre-test check (`BenchTest/sw7_check.py`) reports each link: telemetry from the ESP32, the three time-of-flight readings, the camera rate and the Raspberry Pi 4's power and temperature flags.

Appendix D

Test Evidence

This appendix indexes the raw evidence behind the bench results reported in Chapter 3. Every item is the unedited output of the test it names and is kept in the project repository, under `Report/evidence/` or `BenchTest/logs/`; Table D.1 gives each file. Figure B.3 in Appendix B and Figure 4.1 in Chapter 3 belong to the same record.

Table D.1: Index of test evidence.

Date	Test	Result shown	File
25 Sep 2026	Recorded bench session, telemetry unit, 150 s from power-up	200 Hz on both IMUs with no dropped samples; 149 fused windows, all from both sensors; five of five fusion self-tests passed; acquisition gaps of 25 to 30 ms every 12 s	`BenchTest/logs/2026-09-25_bench_fusion_shake.log`
25 Sep 2026	The same log replayed through the bench console (Figure D.1)	Every subsystem's state at the end of the session, and the vibration and disagreement traces for the whole recording	`Report/evidence/2026-09-25_bench_console_replay.png`
28 Sep 2026	Hazard detector, first run on live camera frames	430 frames in 60.1 s (7.2 fps); a person in view logged as "motorcycle", which exposed the label-offset fault	`Report/evidence/2026-09-28_camera_run1_label_bug.txt`
28 Sep 2026	Hazard detector after the label and colour fixes	419 frames in 60.1 s (7.0 fps); person detected at confidence 0.50 to 0.73; 16 alert activations in 47 s for one person in view, which led to clear-side hysteresis	`Report/evidence/2026-09-28_camera_run2_after_fixes.txt`
29 Sep 2026	Firmware v0.5.0 on the telemetry unit: commands, flash-write stall test, MQTT reconnect	With logging on, 33 to 40 ms gaps on both IMUs every 10 s; with logging off, worst gap 5.9 ms; no dropped samples; 47 MQTT publishes with no failures; reconnect in 0.4 s	`BenchTest/logs/2026-09-29_v050_bench_test.log`

Date	Test	Result shown	File
29 Sep 2026	GNSS outdoor fix on a rooftop, firmware v0.5.0, raw replies requested every 15 s	3D fix after 51 s; 17-field reply with decimal degrees; old parser read satellite counts as the position; well-formed replies 2.4 and 5.5 m from the phone's position; replies garbled by the competing query	BenchTest/logs/2026-09-29_gnss_rooftop.log
30 Sep 2026	GNSS outdoor fix with the corrected parser (v0.5.2), raw replies printed by the firmware	3D fix after 105 s from a cold start; 23 to 25 satellites; over 104 fixes, median 2.7 m and 95% within 11.0 m of the phone's position	BenchTest/logs/2026-09-30_gnss_rooftop_v052.log
30 Sep 2026	MQTT recovery from a 60 s radio-off period (AT+CFUN=4, then AT+CFUN=1), v0.5.0 and v0.5.3	Loss detected in 0.8 to 20.7 s; retries at 30 and 60 s; reconnect 36 s after the radio returned in v0.5.0, about 5 s in v0.5.3; no failed publishes afterwards	BenchTest/logs/2026-09-30_signal_loss_test.log, .._v053.log
30 Sep 2026	Sync-pulse edges logged on the Pi (GPIO17, kernel timestamps), two 60 s runs	Run 1: 16 s burst of spurious edges, consistent with a disturbed contact; run 2: 121 edges, median interval 500.06 ms, range 499.72 to 500.41 ms	BenchTest/logs/2026-09-30_sync_edges_run1_noisy.csv, .._run2.csv
30 Sep 2026	Camera detector latency on the Pi, three 60 s runs (4 threads, 4 threads with live view, 1 thread)	Median 102, 105 and 186 ms from sensor readout to result; 19.5, 17.3 and 7.0 frames per second; no throttling	BenchTest/logs/2026-09-30_c4_latency/
1 Oct 2026	Firmware v0.6.0 bench run, 330 s, logging on throughout (buffered log, signal quality, Hall counter in loopback)	Five flushes; all ten gaps over 10 ms (37 to 40 ms) within 3.5 s after a flush; worst gap otherwise 5.9 ms; no dropped samples; Hall counter 2.00 edges/s; CSQ 24 to 26	BenchTest/logs/2026-10-01_v060_bench_test.log
1 Oct 2026	Read-back of the stored telemetry log (firmware v0.6.3, BenchTest/log_dump.py, four passes)	6,020 records recovered complete; 5,545 contained nan (invalid JSON, fixed in v0.6.3); 14 records with a valid BMS reading (74.78 V, 60%) from 30 Sep; log then cleared	BenchTest/logs/2026-10-01_telemetry_log_dump.jsonl, ..clean.jsonl
1 Oct 2026	Stand-in ToF: micromouse sensor board on the Pi (three VL53L0X, tof_probe.py, tof_reader.py)	Chip confirmed VL53L0X; three sensors addressed and read at about 31 per second each; centre 267 to 300 mm, right 24 to 38 mm, left 8190 (no target) throughout	BenchTest/logs/2026-10-01_tof_three_sensors.csv

Date	Test	Result shown	File
1 Oct 2026	Dashboard camera page (laptop, offscreen) against the live detector on the Pi 4, 60 s	Live after 0.6 s; 638 frames decoded in 45 s (14.2 per second); offline state shown when the detector stopped; no alerts (nothing in view)	Report/evidence/2026-10-01_-dashboard_page_-live.png, DashboardIntegration/tests/test_front_-camera_page_-live.py
1 Oct 2026	Dashboard camera page against the live detector with a person in view, 40 s	Red "Person, Immediate" alert listed 0.5 s after start, matching the box on the video; 15.4 frames per second, 109 to 118 ms; clearing not seen (person stayed in view)	Report/evidence/2026-10-01_-dashboard_page_-person.png
1 Oct 2026	Camera colour check (C3): stills with the standard and the NoIR tuning, detector frames with NoIR tuning and saturation 1.8	Standard tuning: white wall pink; NoIR tuning: neutral but washed out (blue card 120, 124, 125); with saturation 1.8: blue 1, 86, 216; red 222, 8, 77; green 6, 124, 103; grey 124, 129, 138; white 186, 199, 197 (slightly cool)	Report/evidence/c3/
2 Oct 2026	First pothole model: YOLO11n fine-tuned in Colab (T4) on a public 608-image pothole dataset, test split at 640 and 320	mAP@50 59.7% (P 0.66, R 0.50) at 640; 55.0% at 320; 46 epochs, 15 min	CameraDetection/training/runs/run1_-gerapothole_640/
2 Oct 2026	First pothole model on the South African test split (605 images, 1,238 potholes) at 640, 1280 and 1920 input; labels checked visually	mAP@50 0.000 at all three sizes, recall at most 1.2%; label positions confirmed correct	Colab session notebook (to be saved)
2 Oct 2026	Firmware v0.7.0 DASH line on the bench (40 s) and through the Raspberry Pi 4 bridge to a laptop	40 lines in 40 s, none missing, all valid JSON, max 272 bytes; IMUs 200 Hz, 0 drops; bridge served live record (0.37 s old) and ToF distances	BenchTest/TEST_-PROCEDURES.md T11, T12
2 Oct 2026	YOLO11n (COCO) via NCNN on the Raspberry Pi 4, 320 input, 4 threads, 15 s trial	7.9 fps; inference 111 ms median; 175 ms sensor to result (p95 258 ms)	CameraDetection/ncnn_bench.py
2 Oct 2026	Dashboard map crash on the Raspberry Pi 5, diagnosed over ssh without changing the team's installation	Segmentation fault about 2 s after the dashboard is shown with the web map (online or offline data); no crash with the native map after 30 s; the web engine alone loads normally	DashboardIntegration/PI5_MAP_CRASH.md

Date	Test	Result shown	File
3 Oct 2026	Dashboard live-only mode and automatic Raspberry Pi 4 finding, offscreen against a simulated bridge (laptop)	With the bridge down every page shows -- or Unavailable and no simulated value; live when the bridge appears, back to no data when it stops; host found by localhost, override and beacon, and followed after it moved	DashboardIntegration/tests/test_live_data_provider.py
4 Oct 2026	Street-tile map, offscreen, fed the generated UCT replay track (not recorded data) against a stand-in tile server	Tiles requested at most two at a time and cached; heading-up follow; "Waiting for GPS fix"; REPLAY badge; vector roads with the tile server down	DashboardIntegration/tests/tile_map_*.png
4 Oct 2026	Camera and time-of-flight fusion, offline tests	29 of 29 pass; not yet run on the Raspberry Pi 4 (test C9 planned)	CameraDetection/tests/test_sensor_fusion.py
6 Oct 2026	Sensors page and hazard banner, whole dashboard offscreen against the live bridge	ToF gauges (ahead 0.43 m red, right 0.69 m amber, left no reading), both listed as immediate; banner over the driving page, hidden on the Sensors page	DashboardIntegration/tests/sensors_page.png, hazard_banner.png
6 Oct 2026	First fused camera and time-of-flight hazard, live, before calibration	Person, ahead sensor: camera 0.874 m, time-of-flight 0.096 m, reported 0.096 m; not checked against a tape	BenchTest/ TEST_PROCEDURES.md, test C9 notes
6 Oct 2026	Pothole model run 2, South African road-band tiles (Colab)	Validation mAP@50 0.714 at 640 and 0.566 at 320; held-out test tiles not yet scored	CameraDetection/training/runs/run2_sa_band_tiles_640/training_record.json
6 Oct 2026	Simulated drive on the street map, offscreen tests	Starts without a fix, 25 km/h along the route, heading-up, arrives, a real fix stops it	DashboardIntegration/tests/tile_map_simulated_drive.png
6 Oct 2026	Raspberry Pi 4 power and temperature watcher, 20 min	Clear with nothing attached; soft temperature limit from 17:45; under-voltage 14 s after the telemetry unit was plugged into its USB port	BenchTest/logs/2026-10-06_pi4_power_watch.log

The two detector runs end with the following lines, copied from the Raspberry Pi terminal. Before the fixes, every alert carried the wrong class:

```
Stopped. 430 frames in 60.1s (7.2 fps average).
{"class": "motorcycle", "distance_band": "immediate", "confidence": 0.66,
 "timestamp": "2026-09-28T11:58:32", "alert_status": "active"}
```

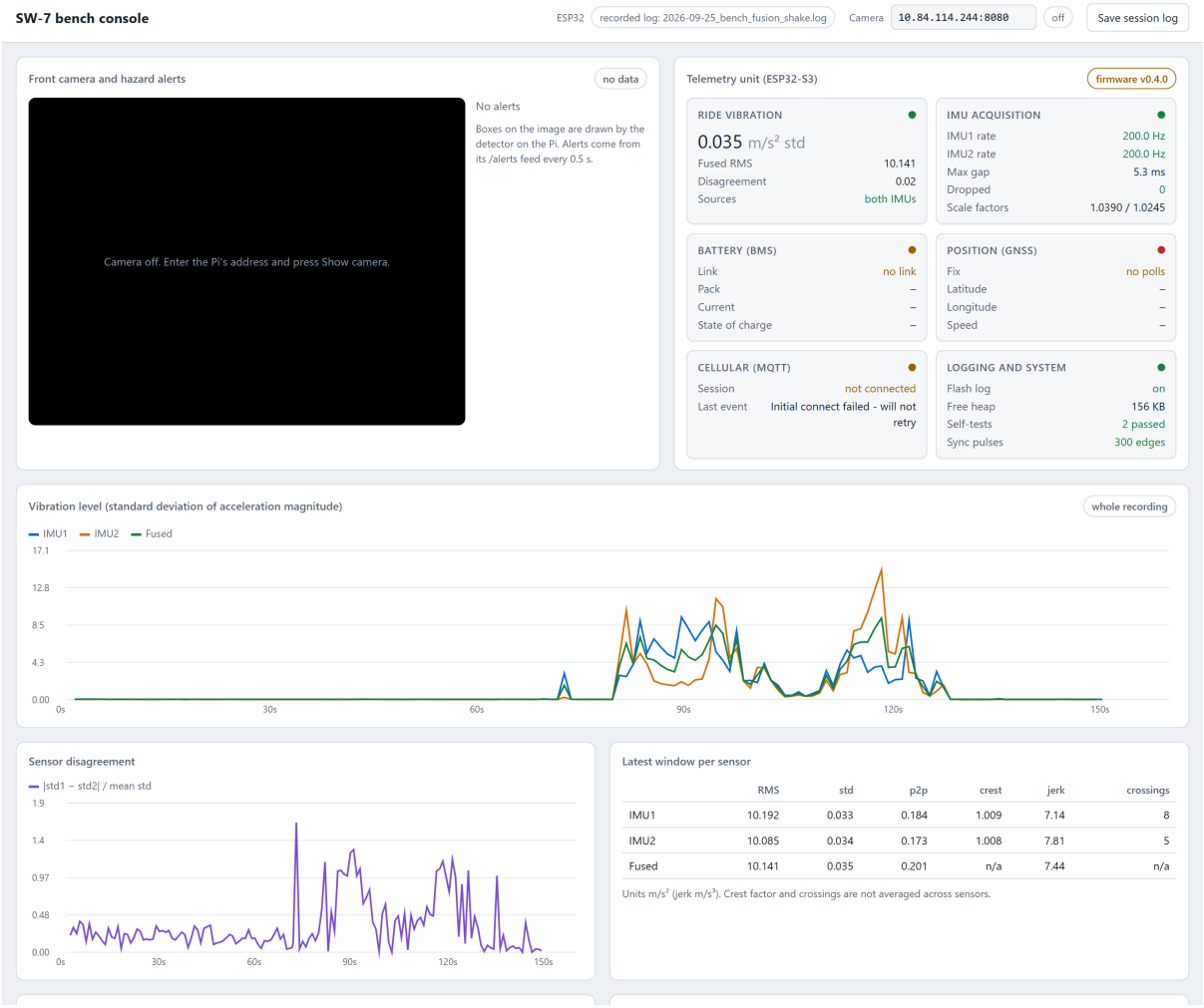


Figure D.1: The recorded bench session of 25 September 2026 replayed through the bench console, which parses the telemetry unit's serial output. Only recorded data is shown; the camera panel is off because the camera was not part of this session.

After the fixes, the class is correct, and the log shows the alert clearing and re-activating within a few seconds while the same person stayed in view:

```
[14:31:28] person band=immediate conf=0.59 streak=3/3
  -> ALERT logged: person (immediate)
Stopped. 419 frames in 60.1s (7.0 fps average).
{"class": "person", "distance_band": "immediate", "confidence": 0.59,
 "timestamp": "2026-09-28T14:31:28", "alert_status": "active"}
{"class": "person", "distance_band": null, "confidence": 0.0,
 "timestamp": "2026-09-28T14:31:30", "alert_status": "cleared"}
{"class": "person", "distance_band": "immediate", "confidence": 0.59,
 "timestamp": "2026-09-28T14:31:31", "alert_status": "active"}
```

Appendix E

Configuration and Development Record

This appendix holds the firmware configuration, the record of development faults, and supporting detail moved out of Chapter 3. The full text of the earlier, chronological Methodology is kept in the project repository (`Report/archive_full_2026-10-11/`).

E.1 Firmware configuration

Table E.1 gives the telemetry unit's configuration as committed for firmware v0.7.3, the last version flashed before this report was cut (11 October 2026). It is taken from `TelemetryUnit/platformio.ini` and `TelemetryUnit/src/main.cpp`; installed package versions are those in the PlatformIO folders on the development laptop, since `platformio.ini` does not pin them. Firmware v0.7.4, in preparation at the time of writing, changes the IMU low-pass filter to 94 Hz and adds sequence numbers for database synchronisation; tasks and queues are unchanged.

Table E.1: Telemetry unit configuration (firmware v0.7.3).

Item	Setting
Build environment	PlatformIO, environment `esp32-s3-devkitc-1`, Arduino framework
Platform and core (installed)	`espressif32` platform 7.0.1; `framework-arduinoespressif32` 3.20017.241212 (Arduino-ESP32 core 2.0.17)
Libraries (installed)	Adafruit MPU6050 2.2.9 (declared ^2.2.4); Adafruit Unified Sensor 1.1.15 (^1.1.14); Adafruit BusIO 1.17.4 (^1.14.1)
Build flags	`ARDUINO_USB_MODE=1`, `ARDUINO_USB_CDC_ON_BOOT=1` (application serial on native USB)
Flash	16 MB, DIO mode; PSRAM memory type left at default (PSRAM not used)
Partition table	`default_16MB.csv`: nvs 0x9000 (20 KB), otadata 0xE000 (8 KB), app0 0x10000 and app1 0x650000 (6.25 MB each), data 0xC90000 (3.375 MiB, LittleFS), coredump 0xFF0000 (64 KB)
Serial	USB CDC at 115200 bit/s; bounded 3 s wait for a host at boot

Item	Setting
IMU1	MPU6050 at 0x68, I^2C bus 0, SDA GPIO8, SCL GPIO9
IMU2	MPU6050 at 0x68, I^2C bus 1, SDA GPIO17, SCL GPIO18
I^2C clock	400 kHz on both buses
IMU settings	$\pm 8\,g$, ±500°/s, low-pass filter 260 Hz; 200 Hz (5 ms) sampling
Calibration	200 samples at rest; factor accepted in 0.5 to 2.0; stored in NVS namespace `imucal`
Modem	A7670X on UART1, RX GPIO47, TX GPIO48, 115200 bit/s 8N1; PWRKEY GPIO4, RESET GPIO5
Sync pulse	GPIO15, toggled every 500 ms
Hall counter	Pulse counter unit 0, both edges, filter 1000 APB cycles (12.5 μs); input on GPIO15 (loopback) until the tap is wired; pole pairs not set
BMS	BLE, service 0xFF00, notify 0xFF01, write 0xFF02; query every 3 s; receive buffer 128 bytes
Periods	GNSS poll 3 s; statistics 2 s; ride window 1 s (200 samples); dashboard line 1 s (at most 480 bytes); MQTT publish 10 s; network position 60 s without a fix; assisted GNSS every 2 h
Fusion	Pair skew at most 0.5 s; fallback after 3 s; σ floor 0.05 m/s^2; speed index needs $v \geq 1$
MQTT	Broker `test.mosquitto.org`:1883, topic `sw7/telemetry`, keep-alive 60 s; retry 30 s doubling to 300 s; session lost after 3 failed publishes
Local log	LittleFS `/telemetry.log`; cap 3 MB; 8 KB memory buffer flushed every 60 s
Tasks (name: core, priority, stack in bytes)	
Acquisition	Imu1Task, Imu2Task: core 1, priority 3, 4096
Statistics, sync	StatsTask: core 1, priority 2, 4096; SyncTask: core 1, priority 2, 2048
Ride features	RideChar1Task, RideChar2Task: core 1, priority 1, 4096
Modem, BMS, dashboard, Hall	ModemTask: core 0, priority 1, 4096; BmsTask: core 0, priority 1, 8192; DashTask: core 0, priority 1, 4096; HallTask: core 0, priority 1, 2048
Queues (depth in items)	
Queues	imu1Queue, imu2Queue: 100 samples each; imu1CharQueue, imu2CharQueue: 400 each; gnssQueue: 20; bmsQueue: 20

E.2 Development faults

Table E.2 lists the faults found during development, with their cause and fix. The faults that shaped the design are also discussed in Chapter 3.

Table E.2: Development faults, causes and fixes.

Symptom	**Cause (how found)**	**Fix and lesson**
Boot loop at a fixed program counter after enabling 16 MB flash and PSRAM	Quad-I/O access requested on flash that runs in DIO mode (one setting changed at a time; ROM banner reports DIO; chip detection confirmed 8 MB PSRAM)	16 MB with DIO mode set together with the partition table; PSRAM setting left at default. Set flash size, mode and table together
No output past the ROM banner on the target board	Serial had been routed to UART0 for the bare development board; on the target board UART0 goes to a separate CH340K port (each hypothesis tested in turn; schematic)	Native-USB flags restored and documented in `platformio.ini`. The two boards need opposite settings
Modem silent after a reset	Power key not pulsed (v0.1); later, the pulse at every boot switched an already running modem off	v0.2.0 pulses GPIO4 for 3 s; v0.5.2 pulses only if `AT` gets no answer
GNSS queue full after about a minute	Statistics task drained the IMU queues but not the GNSS queue	GNSS queue added to the drain cycle
Every GNSS poll rejected, then the first fix at 0.25 and 0.10 degrees	Assumed 13-field reply; real reply 9 fields indoors, 17 or 18 with a fix, decimal degrees (raw replies logged)	Parser anchored on the hemisphere letter, layout and jump checks (v0.5.1 to v0.5.3)
Zero GNSS polls reported for the first minutes	Polls time out for 60 to 90 s after GNSS power-on; statistics count only answered polls	Timeout lines distinguish the cases; allow warm-up before recording
Sync output pin not on any header	Placeholder chosen before the schematic was available (schematic and inspection)	Reassigned to GPIO15 before wiring
No sync edges, later a burst of spurious edges	Jumper disturbed by handling (ESP32 log showed the pulse still running)	Reseated; secured connector needed on the vehicle; off-pattern edges discarded
BMS scan never retried	Scan made conditional on an address set only after the device is found	Scan whenever disconnected and not scanning
BMS connection dropped before reading	Characteristics not under the assumed service in this BLE library (compared with a teammate's parser)	All discovered services searched
Every BMS frame rejected	Checksum included the register byte, so it failed by exactly 3 (the firmware's own query frame shows the rule)	v0.5.4; confirmed by 14 valid records on 30 Sep
MQTT reported connected when it was not; no retry after a failure	First `OK` taken as success; separate result line ignored (modem manual)	v0.5.0 waits for the result code; retry with backoff

Symptom	Cause (how found)	Fix and lesson
Reconnection 36 s after coverage returned	Waited for the next scheduled retry although data were back in 2.4 s	v0.5.3 reconnects on the modem's data-available report (about 5 s)
33 to 40 ms acquisition gaps every 10 s	Flash writes suspend code running from flash on both cores (logging on and off test)	v0.6.0 buffers and flushes once a minute; pause length unchanged
Most stored records not valid JSON	Missing values printed as `nan` (full log read-back)	v0.6.3 writes `null`
Speed index dominated by a constant	Mean square of the magnitude includes gravity	v0.7.2 uses the variance
A person reported as "motorcycle"; poor confidence	Placeholder label shifted every class; camera buffers in BGR order	Label list and channel order corrected (fixed together)
16 alert activations in 47 s for one person	Alert cleared on a single missed frame near the threshold	Clears after 5 frames; nearer band needs 3 frames
Detector slower with the live view on	Drawing and encoding in the detection loop	Moved to a separate thread taking the newest frame
Pink image; later black objects purple	Module has no infrared-cut filter (stills with standard and NoIR tuning)	NoIR tuning and saturation 1.8; dark-pixel correction in the live view only
Raspberry Pi 4 drive would not boot	Boot files copied instead of an image written: one partition, missing root	Rewritten with Raspberry Pi Imager
Raspberry Pi 4 did not join the hotspot; ssh closed	5 GHz hotspot without a regulatory domain; ssh not re-enabled in a second provisioning pass	Hotspot on 2.4 GHz and country code set (sufficient fix not isolated); ssh enabled
Recording aimed at the ceiling	No rigid camera mount	Live preview before recording; printed casing
Bracket model with no lens lip	Cavity cut through the face, visible only in a full render	Cut bounded; checked by full render (bracket not printed)
Simulated speed and battery on the dashboard with no sign	Dashboard fell back to simulation when the bridge address changed	Live-only mode and network discovery
Pi 4 not found on a phone hotspot; 6,828 lookup warnings	1 s probe shorter than the hotspot round trip; searches every 2 s	3 s probe; back-off to 30 s; beacon tried at once
Dashboard crash about 2 s after showing the web map	Native crash in the web engine on the Pi 5	Native map selected, web map not created; Qt tile map
Screen blank after 600 s; poller at 53% of a core	X11 screensaver and power management; game-controller poller	Both switched off by the start script
Team's dashboard started after a reboot	Desktop auto-start is not read on the bare X display	System service for the integrated copy
All three distances null	Start-up stopped at the first sensor that did not answer	Skip, log and re-probe; 0.5 s I/O timeout

Symptom	Cause (how found)	Fix and lesson
Pi 4 under-voltage, throttling, failure to boot	LTE modem powered from the Pi's USB port (2 s power-flag log)	Separate supply for the telemetry unit; fan for temperature
Touch felt slow	Panel reports contact with the first position; buttons act on release (25 taps read directly)	Act on press and hide the pointer (next step)

E.3 Supporting detail

Raspberry Pi 4 discovery

The dashboard's resolver (`DashboardIntegration/sw7_endpoints.py`) tries, in order: localhost, an explicit override, a fixed address on a planned Ethernet link (10.20.0.1 and 10.20.0.2), the source of a UDP beacon sent by the bridge every 2 s on port 50808 (ignored if older than 10 s), `pi4-camera.local` through mDNS, and the last address that worked (Figure E.1). A candidate is accepted when it answers with valid JSON within 3 s; three failed polls start a new search, with the wait doubling to 30 s. The offscreen tests pass; the wired link has a setup script but its use on the vehicle is not recorded.

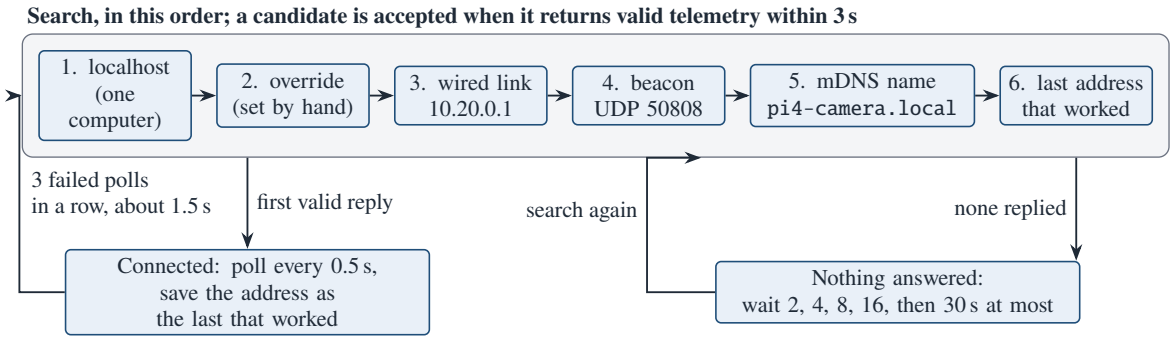


Figure E.1: How the dashboard finds the Raspberry Pi 4 (`DashboardIntegration/sw7_endpoints.py`). The same resolver serves the camera page and the live-data provider.

Camera colour

With the NoIR tuning and saturation 1.8, printed cards held about 1 m from the camera measured as in Table E.3 in the detector's own frames under indoor light. The three colours are clearly identified; grey and white show a small cool shift. Held together in one frame the cards remained distinct. The dark-pixel correction scales each pixel's saturation from fully grey at a brightest channel of 50 or below to unchanged at 110 and above; it took 84 ms a frame with Pillow on the Raspberry Pi 4, so it runs only in the live-view thread and the detector's input is unchanged. The colours should be checked again in daylight.

Table E.3: Colour cards in the detector's frames with the NoIR tuning and saturation 1.8 (mean red, green and blue, 0 to 255; 1 October 2026, indoor light).

Card	R	G	B
Blue	1	86	216
Red	222	8	77
Green	6	124	103
Grey	124	129	138
White	186	199	197

Pothole model

Run 1 (2 October 2026) fine-tuned YOLO11n, pretrained on COCO, on a public 608-image pothole dataset in Colab: 46 epochs, stopped 15 epochs after the best validation result; mAP@50 59.7% at 640 pixels (precision 66%, recall 50%) and 55.0% at 320 on the dataset's own test images. Validation loss stopped improving after about 20 epochs, so more varied images were needed, not longer training. On the South African dataset's 605 test images (1,238 potholes) it scored 0.000 at 640, 1280 and 1920 pixels, recall at most 1.2%; drawing the converted labels on an image confirmed that the label conversion was correct. Of that dataset's 4,337 images (3680×2760 pixels), 1,987 contain 5,029 potholes, with a median size of 100 by 26 pixels, about 17 by 4.5 pixels after reduction to a 640-pixel input, and all centres lie between 48 and 62% of the image height. Run 2 (6 October) cropped each frame to the band from 42 to 68% of its height and cut it into three overlapping 1280-pixel tiles, so that a median pothole is about 50 by 13 pixels at the training input; after 60 epochs on a T4 it reached a validation mAP@50 of 0.714 at 640 pixels (precision 81%, recall 62%) and 0.566 at 320. These are validation scores, optimistic because validation chose the best epoch, and per tile. On the Raspberry Pi 4, a COCO YOLO11n through NCNN at a 320-pixel input ran at 7.9 frames/s, 175 ms median from readout to result (`CameraDetection/ncnn_bench.py`).

Dashboard map and navigation

The street-tile map (`DashboardIntegration/sw7_tile_map.py`) draws OpenStreetMap tiles with Qt's painter under the tile usage policy (at most two requests at a time, an identifying user agent, no bulk download), caches tiles for 30 days, falls back to scaled lower-zoom tiles or the team's vector roads, and follows heading-up at zoom 17. A prefetch tool cached 656 tiles (11.9 MB) at zoom 11 to 16 around the campus. Routes come from the public OSRM service. A generated replay track and a simulated drive allow the map to be shown indoors; both are labelled on screen (REPLAY, Simulated drive) so that they cannot be taken for real positions, and a real fix stops the simulation. The Raspberry Pi 5 also carries a Hailo-8 accelerator, which did not appear on the PCI Express bus because the external connector is not enabled in its boot configuration.